

USEFUL MEMORIES BECOME FAULTY WHEN CONTINUOUSLY UPDATED BY LLMs

Dylan Zhang¹, Yanshan Lin², Zhengkun Wu², Yihang Sun¹, Bingxuan Li¹, Dianqi Li and Hao Peng¹

¹University of Illinois Urbana-Champaign, ²IIS, Tsinghua University, Work done at UIUC

Learning from past experience benefits from two complementary forms of memory: *episodic* traces—raw trajectories of what happened—and *consolidated* abstractions distilled across many episodes into reusable, schema-like lessons. Recent agentic-memory systems pursue the consolidated form: an LLM rewrites past trajectories into a textual memory bank that it continuously updates with new interactions, promising self-improving agents without parameter updates. Yet we find that such consolidated memories produced by today’s LLMs are often faulty even when derived from useful experiences. As consolidation proceeds, memory utility first rises, then degrades, and can fall below the no-memory baseline. More surprisingly, even when consolidating from ground-truth solutions, GPT-5.4 fails on 54% of a set of ARC-AGI problems it had previously solved without memory. We trace the regression to the consolidation step rather than the underlying experience: the same trajectories yield qualitatively different memories under different update schedules, and an episodic-only control that simply retains those trajectories remains competitive with the consolidators we test. In a controlled ARC-AGI Stream environment that exposes RETAIN, DELETE, and CONSOLIDATE actions, agents preserve raw episodes by default and double the accuracy of their forced-consolidation counterparts; disabling consolidation entirely (episodic management only) matches this auto regime. Practically, robust agent memory should treat raw episodes as first-class evidence and gate consolidation explicitly rather than firing it after every interaction. Looking forward, reliable agentic memory will require LLMs that can consolidate without overwriting the evidence they depend on.

1. Introduction

Memory consolidation is the cognitive process by which individual experiences are stabilized, reorganized, and integrated into long-term memory over time. Rather than preserving every episode as a raw record, consolidation helps extract recurring patterns, discard incidental details, and connect new information with existing knowledge. A central hallmark of intelligence is the ability to turn repeated experience into reusable knowledge. Humans do not treat every new situation as entirely unfamiliar; instead, they gradually summarize common structure from past episodes and organize it into abstractions through memory consolidation (McClelland et al., 1995; Squire et al., 2015; Dudai et al., 2015; Moscovitch et al., 2016; Sun et al., 2023; Bartlett, 1932). These consolidated abstractions allow humans to transfer lessons learned in one context to novel but related situations, enabling efficient reasoning, faster adaptation, and more robust generalization beyond the exact events they have previously encountered.

Recent agentic-memory systems pursue a similar idea for LLM agents. After solving a problem, an agent can distill its trajectory into a textual abstraction of the skill it has learned, store this abstraction in persistent memory, and refine it as more related problems are encountered (Shinn et al., 2023; Zhao et al., 2024; Zhang et al., 2025; Sumers et al., 2024; Park et al., 2023; Majumder et al., 2023; Wang et al., 2023, 2024; Li et al., 2026; Ouyang et al., 2026). This paradigm is especially appealing because it suggests a practical path toward self-evolving agents: Experience can be accumulated, compressed, and reused as compact textual knowledge, without requiring any parameter updates.

In this work, we systematically examine what happens when agents repeatedly compress past trajectories into memory and reuse that memory over long horizons. To make the experiences themselves informative by construction, we study settings in which the input trajectories are demonstrably useful: either drawn from problems the agent has already solved without memory, or supplied with ground-truth solutions. Even under these favorable conditions, we find that **current LLMs are not reliable memory consolidators**. Each con-

solidation step is a lossy rewrite of the memory store: useful details are dropped, spurious rules are introduced, and once-helpful abstractions drift away from the underlying task structure. Across ALFWorld, ScienceWorld, WebShop (Yao et al., 2022), AppWorld (Trivedi et al., 2024), and a controlled testbed we introduce to make consolidation failures more visible, agents that incrementally abstract their accumulating trajectories into a textual memory bank can briefly improve or plateau before degrading—in some cases falling below the same agents without memory at all (Fig. 1a). The clearest case isolates the consolidation step from any input-side excuse: GPT-5.4 first solves a set of ARC-AGI problems at 100% accuracy with no memory; after consolidating from ground-truth solutions to those very problems, it then fails on 46% of them (Fig. 2). The failure can therefore be attributed to **faulty memory** rather than to noisy trajectories.

Consolidation is fragile in a second sense: the same set of trajectories can yield qualitatively different memories depending on the order and grouping of updates. Consolidating the whole trajectory pool in one pass (**Static-All**) versus streaming it batch-by-batch (**Stream**) produces different end states (Fig. 3); updates on one task overwrite memory of another (Figs. 8b and 8c); and a stream of repeated near-duplicates causes the memory to overfit to seen instances and generalize poorly within the same task (Fig. 11). Meanwhile, an episodic-only control that consumes the same trajectories without abstracting them—appending raw rollouts to context as in-context demonstrations—is already competitive with the lesson-style consolidators we test (Table 2). Because the trajectory pool is held fixed across these comparisons, the variance across schedules and the episodic-only control together point to the consolidation step itself, not the underlying experience, as the source of the failure.

We further identify three mechanisms behind faulty memory (§6). First, agents misgroup experiences before abstracting them, pooling episodes that do not share underlying structure. Second, even when grouping is correct, abstraction can strip the applicability conditions of a lesson, so that overgeneralized entries interfere with neighboring tasks. Third, when the input stream is narrow, abstraction overfits to seen instances. Together, these failure modes weaken the boundary between what should be generalized, what should remain task-specific, and what should be preserved as raw episodic evidence (Sun et al., 2023; Bartlett, 1932; Brainerd and Reyna, 2002).

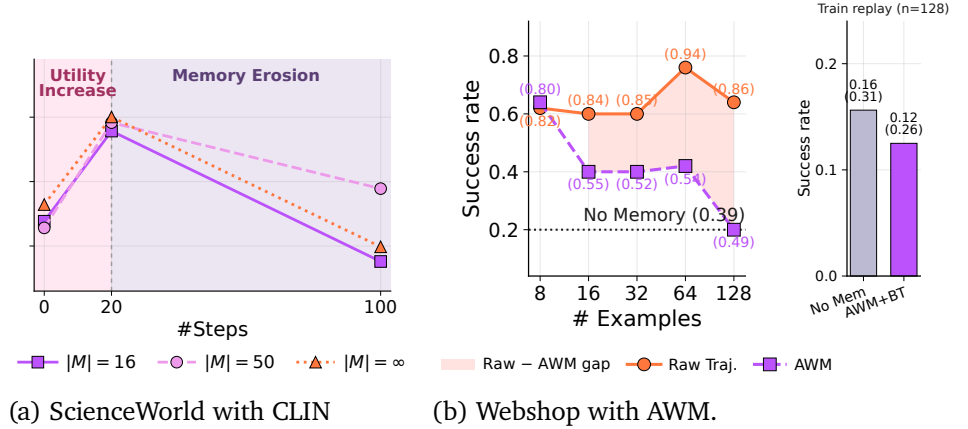


Figure 1: Memory utility degrades as consolidation continues. (a) ScienceWorld utility rises early then declines across memory sizes. (b) On WebShop, AWM-distilled memory degrades with scale while raw trajectories remain robust. The y-axis is success rate (number solved out of 50 held-out episodes); the parenthesized value next to each marker is the average score, a separate WebShop metric.

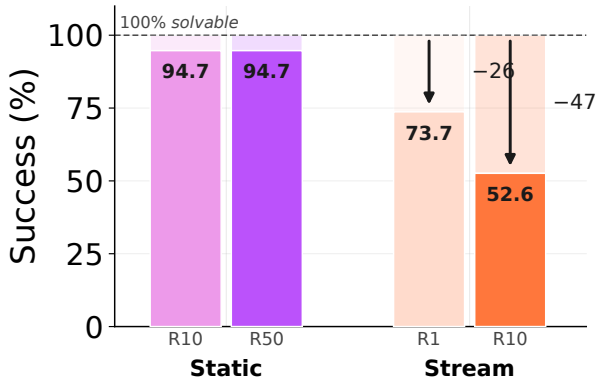


Figure 2: Iterated stream consolidation collapses on solvable tasks. GPT-5.4 accuracy on a 19-problem ARC-AGI slice it solves at 100% without memory (dashed ceiling). Since ground-truth solutions are available at every consolidation step, all input trajectories are useful by construction. We compare *Static*, which consolidates the full pool in one update, with *Stream*, which updates memory after each problem. Static remains at the ceiling after $R = 10$ and $R = 50$ refresh rounds, whereas Stream falls to 52.6% by Round 10 on the same previously solved problems.

These findings suggest a limitation in current LLM agents: while they can often benefit from individual past episodes, they do not reliably turn those episodes into stable and reusable abstractions over long horizons. Many recent agentic-memory systems—CLIN (Majumder et al., 2023), Agent Workflow Memory (Wang et al., 2024), Dynamic Cheatsheet (Suzgun et al., 2026), and ACE (Zhang et al., 2025)—adopt update-after-every-interaction designs that treat each consolidation step as at worst neutral; our results contradict this assumption. For today’s models, retaining episodes is the more robust default: when ARC-AGI Stream agents are given retain, delete, and consolidate actions, they preserve episodes by default and outperform their forced-consolidation counterparts. Yet episodic accumulation cannot be the long-term answer: as deployment continues, raw histories grow unboundedly and resist compositional reuse—compression and transfer ultimately require abstraction. We see faulty memory as a central open problem in agentic memory, and our results motivate architectures that keep the episodic and abstraction-forming roles distinct rather than collapsing both into a single rewrite loop.

2. Background and Related Work

Memory in LLM agents. LLM agents augment fixed-weight models with editable textual memory that they themselves maintain. Across architectures, this memory takes many forms: hierarchical context stores (Packer et al., 2023; Letta, 2024), reflective notes (Shinn et al., 2023), episodic streams (Park et al., 2023), distilled lessons (Zhao et al., 2024), skill libraries (Wang et al., 2023), causal abstractions (Majumder et al., 2023), dynamic memory stores (Chhikara et al., 2025; Xu et al., 2025; LangChain AI, 2024), evolving playbooks (Zhang et al., 2025), dynamic cheatsheets (Suzgun et al., 2026), and reasoning memory banks (Ouyang et al., 2026). A popular paradigm is that LLMs abstract useful procedural information from experience into text. This work mainly concerns that category.

Schema formation and metacognitive control. Cognitive theories of memory describe two com-

Setting	Figure(s)
ALFWORLD	
GT expert trajectories (128 per type, 6 types).	Figs. 3, 4
SCIENCEWORLD w. GPT5.4-mini	
Offline task stream, batch size 4.	Fig. 1a
Self-solve task-switch sequence.	Fig. 10
WEBSHOP w. GPT-5.4mini	
AWM, 50-episode held-out eval.	Fig. 1b
APPWORLD	
AWM with trajectories from AgentGym.	
ARC-AGI	
Subsets For Individual Experiments.	Figs. 11, 8b, 8a, 2
Stream: offline GT-solution; eval on 50 held-out tasks (same family, unseen).	Figs. 9, 7; Tab. 4
Stream: self-solve, 400 steps.	Fig. 5

Table 1: Settings used across the paper. The right column points to the figure(s) and table(s) each setting produced.

plementary processes: a fast episodic system that stores individual experiences, and a slow process that extracts compact *schemas*—abstractions over many episodes that integrate with prior knowledge to support future generalization (McClelland et al., 1995; Squire et al., 2015; Dudai et al., 2015; Moscovitch et al., 2016). Schema formation is beneficial when new evidence fits existing structure (Tse et al., 2007), but harmful when it is non-selective or poorly controlled: nonselective consolidation causes interference and loss of specificity (McClelland et al., 1995; Sun et al., 2023). Whether consolidation helps therefore depends on *metacognitive control*—deciding which experiences belong together, how abstractly to rewrite them, and when to preserve distinctions rather than collapse them. Human metacognition is itself imperfect (Flavell, 1979; Nelson and Narens, 1990; Koriat, 1997), and whether LLMs reliably monitor their self-generated abstractions is an open question—in adjacent settings, chain-of-thought explanations have been shown to diverge from the underlying computation (Turpin et al., 2023). Agent-memory systems implement an explicit analogue, abstracting “lessons” that are reused on later tasks while putting the same model in charge of both generating the memory and monitoring its own abstraction; the failure modes we observe (§6) are consistent with this control loop being unreliable in practice.

3. Experiment Set-up

We study how models abstract memory from experience across clean and widely-used agent benchmarks—ALFWorld, ScienceWorld, WebShop, AppWorld (Trivedi et al., 2024), and Mind2Web (Deng et al., 2023)—and a synthetic stream we built, *ARC-AGI Stream*, to introspect the memory-management process under known task families and ground-truth solutions.

We adopt representative agentic-memory methods that distill lessons from past trajectories—CLIN (Majumder et al., 2023), Agent Workflow Memory (Wang et al., 2024), Dynamic Cheatsheet (Suzgun et al., 2026), and ACE (Zhang et al., 2025).

Memory construction conditions. Three regimes differ only in how trajectories are presented to the memory manager: **Static-All** abstracts memory from the entire pool in one-shot; **Static-Group** does so for each task type (for ALFWorld and ScienceWorld) and concatenate: this simulates offline memory construction and is used for control; **Stream** updates in batches, simulating an agent that evolves continually. For Stream we change batch size and within-batch task composition (homogeneous vs. heterogeneous) and consider both unbounded and size-capped memory banks. At evaluation we provide all memory entries in context and average two repeats per question unless noted otherwise.

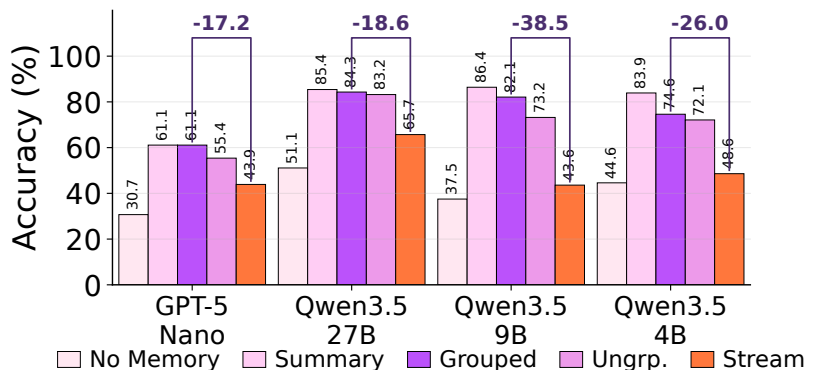


Figure 3: Streaming consolidation collapses. *Stream* loses 17–38 pts vs. whole-batch *Pool*.

ARC-AGI Stream. We introduce *ARC-AGI Stream*, a controlled testbed for introspecting the abstraction process itself rather than only its downstream accuracy. Building on ARCGEN (Moffitt, 2025), the testbed combines three properties that, jointly, are absent from existing agent benchmarks: (i) a fully specified latent task taxonomy—following ConceptARC (Moskvichev et al., 2023), each concept is a distinct problem type, so memory-management actions (Fig. 7a) and results (Fig. 6) can be traced family-by-family. (ii) programmatic ground-truth and (iii) an exposed, structured vocabulary of memory operations whose use we can audit per step. These controls make abstraction errors attributable: if a model receives correct episodes from known families but later merges families, the failure lies in memory abstraction rather than others. Together they enable the family-coverage, misclassification, and buffer-composition analyses in §6 that would not be measurable on ALFWorld/ScienceWorld/WebShop. Following the Complementary Learning Systems framing (McClelland et al., 1995; Sun et al., 2023; Tse et al., 2007), we maintain two stores: an **Episodic** buffer of problem–solution pairs and an **Abstract** store of consolidated lessons; the control conditions below are designed to test this gating prediction directly. At each step the model may **RETAIN** a raw episode, **DELETE** an entry, or **CONSOLIDATE** buffered episodes into the Abstract store; the solver can be conditioned on either store or both stores. We compare three control loops: **Force** requires consolidation each round and no Episodic entries persist between rounds; **Auto** lets the model choose whether to retain raw or abstract; **Episodic Management Only** allows only retention and deletion, disabling abstraction entirely. Two trajectory regimes complete the design: **GT**, where ground-truth solutions are streamed to simulate training on demonstrations, and **Running**, where the agent learns from its own attempts as it solves new problems. Prompt templates are documented in Section B; the family and skill taxonomy used by the generator is in Section A.

4. Useful Memories Become Faulty When Continuously Updated by LLMs

4.1. Stream Updates Reveal The Cumulative Effect Of Imperfect Abstraction

Abstracted memory utility is non-monotonic in updates. Starting from an empty memory store, abstracted-memory utility rises early and then falls below the no-memory baseline as updates accumulate. Figure 1a shows this curve on ScienceWorld with batches of 4 trajectories: the score peaks near step 20 and declines through step 100 for every memory size and sometimes below no-memory baseline. The same shape recurs on WebShop, where AWM goes from 0.64 at 8 examples to 0.20 at 128 while no-memory sits at 0.20 (Fig. 1b)—scaling the memory eventually erases its own benefit.

A strong memory is not a fixed point. A high-quality starting memory does not stabilize under continued consolidation either. We first consolidate ALFWorld trajectories with GPT-5.4—the strongest model we test—following the **Static-Group** setting from §3, then continue updating that store in batches with GPT-5-nano on the same trajectory pool. Evaluated under three solvers (Qwen3.5-{27B, 9B, 4B}) to avoid idiosyncrasy, utility declines across all three solvers and can drop drastically between consecutive evaluation steps (Fig. 4).

The agent regresses on tasks it has already solved. The cleanest, most striking instance is in-distribution regression on a 19-problem slice of the ARC-AGI Stream that GPT-5.4 solves at 100% accuracy without memory. Streaming those same problems through the consolidation loop, with ground-truth solutions available at every step, brings GPT-5.4 down to 54% on the very problems it had previously solved (Fig. 2).

The consolidation procedure itself shapes the rate of decline. The decline is not driven by content alone; how trajectories are presented to the consolidator changes the resulting memory in content-independent ways.

1. **Grouping by task family helps.** Holding update strategy fixed, consolidating each task family separately (**Static-Group**) outperforms consolidating across families jointly (**Static-All** - marked as Ungrp.); pooling heterogeneous tasks into a single abstraction step invites confusion between unrelated procedures (Fig. 3).
2. **One-shot consolidation beats streaming.** Holding the trajectory pool fixed, **Streamed** incremental updates underperform a single whole-pool pass (Fig. 3). This suggests that early abstractions can anchor later rewrites and mistakes are reused as context for future consolidation, so small errors in segmentation or abstraction compound into progressively distorted memories. However, in continual learning loops, memory gets repeatedly updated as agents collect more experiences, instead of seeing the entire pool of past interactions, precisely falling to the lossy update regime.

3. **Heterogeneous batches accelerate the decline within streaming.** Even when updates must be streamed, batches that mix tasks produce worse memory than batches drawn from one task at a time (Fig. 4): the model merges incompatible lessons inside a single update.

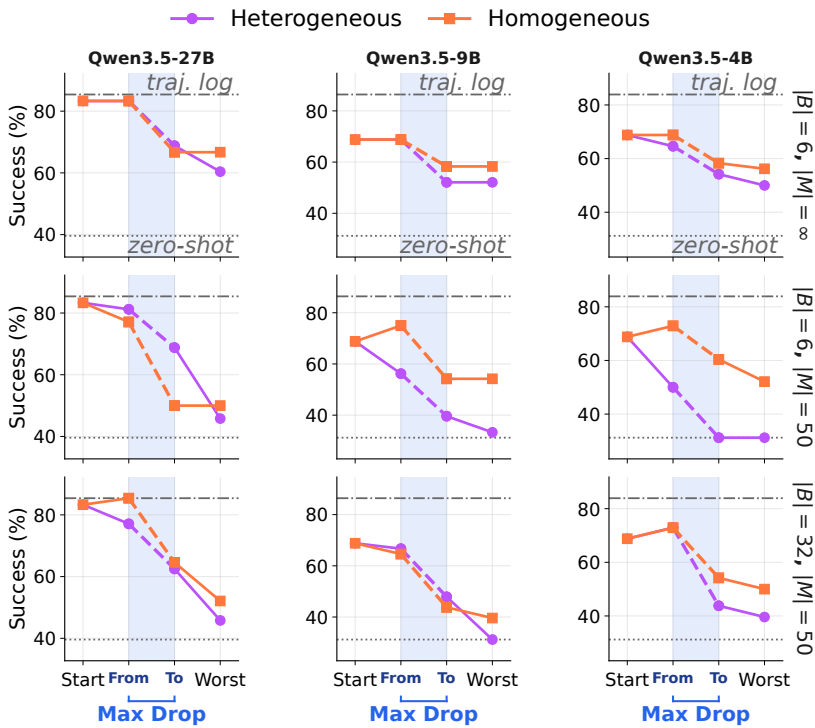


Figure 4: Heterogeneous batches accelerate erosion.

the same experience that ACE, AWM, and Dynamic Cheatsheet are asked to compress, and all memory entries are placed in context at evaluation.

In most cases, abstracted memory cannot significantly outperform direct in-context learning from trajectory logs (or subsets of them). It means the evidence available before abstraction already contains task-relevant signal that the solver can exploit directly: observations, actions, intermediate failures, and environmental

Implications for memory design. Each abstraction pass is a lossy rewrite that compounds over updates. The pattern makes two predictions for how memory should be organized, which §5 tests directly. (i) The raw trajectory logs that the consolidator distills from should remain a competitive baseline, since they retain what each rewrite discards. (ii) Allowing an agent to keep raw episodes alongside (or instead of) abstracting them should outperform a regime that forces every trajectory through consolidation.

4.2. An episodic-only store should be a diagnostic baseline

In WebShop, ALFWorld and AppWorld, an *episodic-only* memory—append-only trajectory rollouts in context, with no cross-trajectory abstraction—is competitive with lesson-style consolidator we test (Table 2; WebShop in Table 3 in Appendix). The trajectory baseline stores

Trajectory Logs (Summarized)												
Backbone	NoMem	All	FS=64	FS=32	FS=16	Incr. Keep	One-Pass Keep	ACE _{GT}	ACE	AWM	DC	
<i>AppWorld</i> (TGC%)												
Qwen3.5-27B	66	73	66	65	68	69	69	76	65 (−)	68 (−)	68 (−)	
GPT-5-Mini	14	23	15	20	16	15	19	10 (−)	13 (−)	19 (−)	8 (−)	
GPT-5.4-Mini	52	59	58	58	54	64	57	57 (−)	52 (−)	52 (−)	52 (−)	
Claude-Haiku-4.5	68	66	69	69	70	70	59	60 (−)	74	68 (−)	64 (−)	
<i>ALFWorld</i> (Task Success %)												
GPT-5.4-Mini	54	90	92	81	77	85	81	85 (−)	79 (−)	65 (−)	58 (−)	
GPT-5-Mini	52	81	81	77	67	75	65	56 (−)	60 (−)	48 (−)	50 (−)	
Claude-Haiku-4.5	79	92	85	77	73	79	90	83 (−)	73 (−)	81 (−)	81 (−)	

Table 2: We report consolidated memory’s effect against the raw trajectories they abstract from. A red (−) marks any non-trajectory-log approach that is beaten by at least one trajectory-log baseline in the same row. Best per row in **bold**. WebShop results are reported separately in Table 3.

feedback remain tied to the concrete situation in which they occurred. Schema-like behavior can emerge from in-context learning on preserved instances comparative to a separately stored abstraction (Hintzman, 1986; Brooks, 1978; Medin and Schaffer, 1978; Logan, 1988). Therefore, we suggest that a memory method whose value depends on distillation should be tested against the unabridged rollouts it distills.

5. Mitigating Faulty Abstraction

The findings in § 4 imply experiences contain information the LLMs can consume for better task performance, but the challenge is that those information fades with repeated LLM updates instead of being refined.

Therefore, the memory architecture should be allowed to preserve episodes instead of forcing every new experience through consolidation. In this section, we present a controlled study to test whether granting models autonomy of retention versus abstraction, and allowing parallel stores of abstract and episodic memory would be a mitigation to the effect.

ARC-AGI Stream lets us instantiate the two-process design explicitly. The agent maintains both an **Episodic** buffer and an **Abstract** store, and at each step may RETAIN, DELETE, or CONSOLIDATE (**Auto**; cf. § 3). We compare this to **Force**, where consolidation is required at every round and no episodic entry persists between rounds. Three observations follow.

First, the episodic store carries most of the gain. Removing

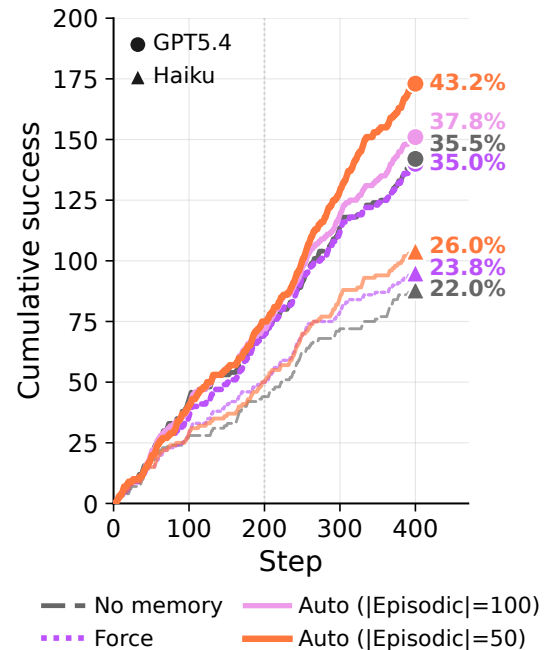


Figure 5: Cumulative success on ARC-AGI Stream across 400 training steps. Auto with a raw-episode buffer matches or beats both *Force* (forced abstraction of every step) and the *No memory* baseline on both backbones.

episodic evidence and using only abstract lessons collapses accuracy back to the no-memory baseline (Fig. 9a), indicating that the useful information was retained in raw episodes rather than distilled entries.

Second, *Episodic Management Only*—retention and deletion of raw episodes with abstraction disabled—matches or exceeds the full Auto mode (Fig. 9a). In this regime, the agent’s gains do not require an abstract store at all.

Third, forced abstraction underperforms retained-episode policies. Across 400 training steps, Auto—which keeps episodes by default and uses abstraction sparingly—generally outperforms Force on both backbones we test (Fig. 5). Whatever Force gains from compression, it can lose more by overwriting evidence.

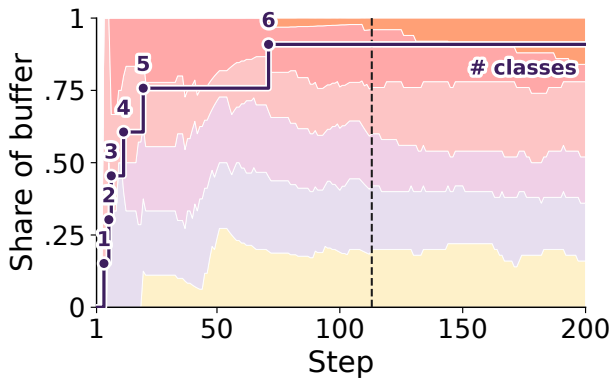


Figure 6: Buffer composition over time on the ARC-AGI Stream

The agent’s own management decisions point in the same direction. Given the option, it saturates the episodic buffer quickly at both budget levels while keeping the abstract store sparse (Fig. 7b). The Auto-mode result therefore reflects an episodic-first policy that the agent chooses when the architecture permits it.

Principle. Episodic and schema-forming roles should not be collapsed into a single rewrite loop. Complementary Learning Systems make this prescription explicitly: a fast episodic store and a slow schema-forming store are kept architecturally distinct so that fast learning does not overwrite the slow one, and consolidation is gated by schema fit rather than triggered on every event (McClelland et al., 1995; Kumaran et al., 2016; Tse et al., 2007;

Sun et al., 2023). The Force regime collapses this separation into a mandatory rewrite loop, recreating the interference conditions the dual-system design was meant to prevent (McCloskey and Cohen, 1989; French, 1999). An episodic store should therefore retain the evidence; abstraction, when it happens, should be opt-in and gated by the agent rather than forced on every trajectory.

In our experiments, both policies that gate abstraction (Auto) and policies that disable it entirely (Episodic Management Only) match or exceed forced abstraction (Force), pointing to *mandatory rewriting at every step*—rather than abstraction itself—as the decisive failure mode in this regime (Nelson and Narens, 1990; Koriat, 1997).

6. Failure Modes

We introspect three abstraction failures both quantitatively and qualitatively: misgrouping before abstraction, interference from overgeneralized lessons, and overfit to narrow streams of examples. We present full case study results in Section D.

6.1. Correct Segmentation Takes Time, Forced Abstraction Breaks It

Correct segmentation—identifying which experiences belong together before abstraction—is a precondition: an agent cannot abstract a lesson from a set of episodes it has failed to group correctly (Zacks et al., 2007).

The capacity for correct segmentation is in fact **present**. Given autonomy over its memory, the model eventually converges to a clean episodic store covering each of the 6 problem types (Fig. 6). Yet the process

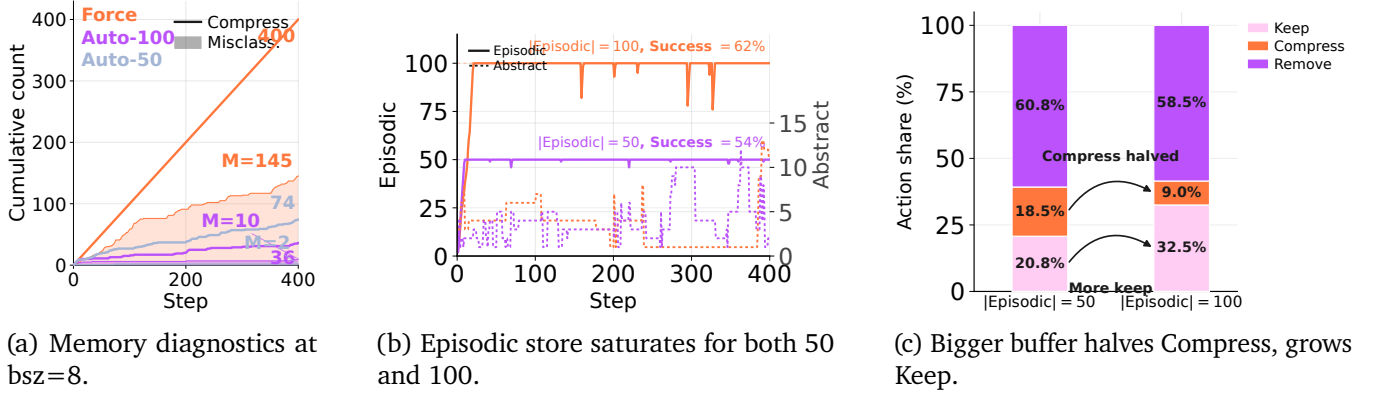


Figure 7: Auto-mode diagnostics on ARC-AGI GT Stream. (a) Compression budget vs. misclassifications. (b) Episodic store saturation under different caps. (c) Distribution of meta-level memory actions as the buffer grows.

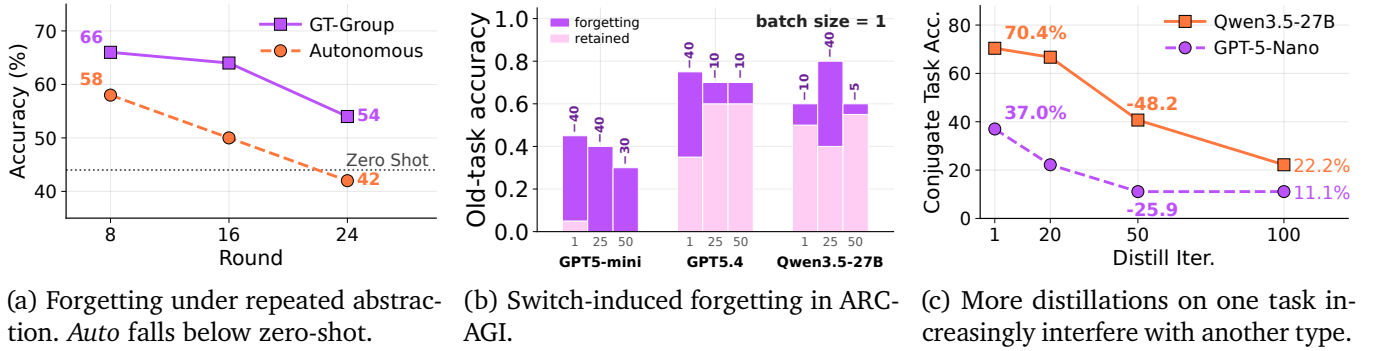
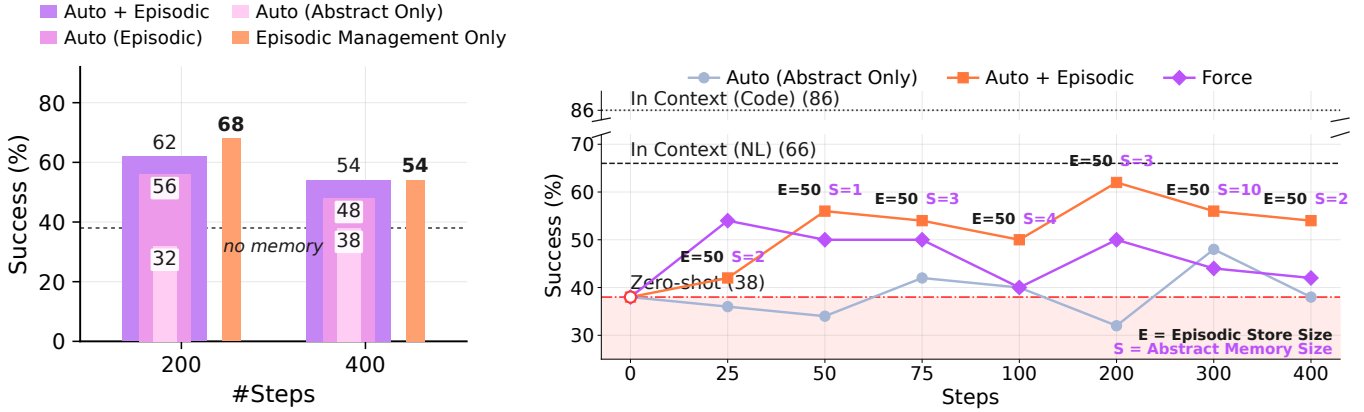


Figure 8: Three views of consolidation-induced forgetting. (a) Repeated consolidation on the same pool erodes utility round by round. (b) A single task switch is sufficient to overwrite a non-trivial fraction of what was previously learned. (c) More distillations on one task increasingly interfere with another type.

takes time: only after 71 steps (568 examples) does the model admit all six problem types into the buffer. In the Force regime, where the model must consolidate at every step (8 fresh examples), it makes mistakes frequently. Figure 7a reports the *misclassification count* — the number of memory-management actions that combine episodes from two or more distinct problem classes — and shows that, under forced consolidation, the model routinely merges across class boundaries. The bottleneck is not recognizing class structure but preserving it through cross-episode abstraction: the forced act of abstraction overrides the segmentation the model is otherwise capable of.

6.2. Interference

Each abstraction pass smooths and abstracts existing entries—a process analogous to the semanticization of episodic memories under repeated retelling (Bartlett, 1932); when the underlying chunks are imprecisely bounded, this abstraction strips the preconditions under which a lesson applies, leaving what reads as broadly relevant when it is not. On a related but different problem (Pick&Place versus Pick-Clean-Place in ALFWorld), the agent sees these overgeneralized lessons and is misled rather than aided (Fig. 8c).



(a) In **Auto**, using the episodes it retained alone shows strong performance, and it synergizes with the abstract store. Meanwhile, managing episodic memory alone beats or matches the performance.

(b) **Training-time success across memory configurations.** Two in-context references — raw code (*In Context (Code)*, 86%) and natural language (*In Context (NL)*, 66%) are in-context learning results on all problems of the same family.

Figure 9: Forced abstraction underperforms retained episodes. (a) Across memory-source ablations, episodic management matches or beats every consolidating mode. (b) On the GT-400 stream, agents solve more problems with retained raw episodes than with forced abstractions.

The same interference dynamic shows up at the entire learning run level under task switching. On the 15-task ScienceWorld switch sequence, we compare two abstraction set-ups at each task t : *Fresh*, which distills memories only on task t , against *Cumulative*, which consolidates jointly over tasks $1..t$. Cumulative ends +203 points behind Fresh (Fig. 10). We use LLM judge to label each memory entry and find Cumulative accumulates over-generalized memories at $\sim 5\times$ Fresh’s rate and garbage memories at $\sim 20\times$, with both gaps widening monotonically across the 15 tasks (App. K.2, Fig. 18).

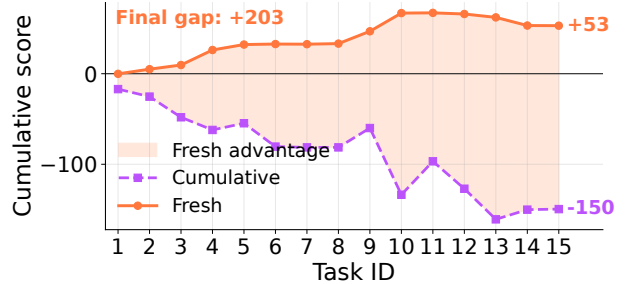


Figure 10: Per-task vs. accumulated consolidation on ScienceWorld. Every additional task folded into a single store erodes the parts that supported the earlier ones.

6.3. Overfit

A complementary failure mode emerges when the input distribution narrows rather than widens. In our ARC-AGI experiments, we repeatedly feed the model tasks drawn from a single strategy family across consolidation cycles. As the iteration count grows, the memory overfits to the specific instances it has seen: performance remains stable on exact repetitions of those problems but collapses on simple variations within the same strategy family (Fig. 11; see Section D for case studies).

The abstraction loop, given a narrow stream, abstracts toward the surface regularities of the observed instances rather than the underlying problem-solving strategy.

Case Study We ground the failures above in concrete memory artifacts across four environments: ARC-AGI strategy-memory pathologies (Section D), ALFWorld memorycollapse (Section G), WebShop AWM collapse-

by-accretion (Section H.1.2), and ScienceWorld abnormal-memory categories (Section J.1). A *memory surgery* ablation isolates their content-level bite: removing a single workflow (W8) from a WebShop consolidated memory raises pass rate (Section I.1), since W8 biases the agent toward `click[Next >]` dead loops at the expense of `click[Buy Now]`.

7. Conclusion

Persistent memory is meant to let LLM agents move beyond static competence: experience is stored, compressed into reusable lessons, and carried forward. We identify an issue that may undermine this promise — across agent benchmarks and ARC-AGI Stream, continuously updated textual memory can become less useful as experience accumulates; in the cleanest case, an agent becomes worse on the very problems its memory was built from.

The failure lies in the consolidation loop itself. Current LLMs do not reliably decide which episodes belong together, which distinctions should survive compression, or when an experience should remain episodic. They misgroup trajectories, overgeneralize lessons, and overfit to narrow streams. Because each update rewrites the products of earlier updates, small abstraction errors compound into faulty memory.

These findings suggest that raw episodes should be treated as first-class evidence, not disposable material to be compressed away. Abstraction should be selective, delayed, and grounded in recoverable trajectories. Until agents can control when and how to consolidate experience, continuously updated textual memory should be treated not as a reliable engine of self-improvement, but as a fragile mechanism that can make more experience produce worse memory.

8. Limitations

Our study has several scope conditions. First, we evaluate text-based agentic benchmarks (ALFWorld, ScienceWorld, WebShop, AppWorld, Mind2Web) and a controlled ARC-AGI Stream environment; whether the same erosion dynamics appear in embodied, multi-modal, or tool-rich production settings remains open. Second, we study natural-language abstraction as implemented by contemporary LLMs (GPT-5.4 family and Qwen3.5 family); parametric memory (weight updates, distillation into the model) and structured non-textual memory representations are out of scope. Third, the consolidator and the solver are themselves LLMs, so faulty abstraction reflects current model capability and could shift with stronger consolidators or with consolidator-specific fine-tuning. Fourth, due to API-cost constraints we report point estimates from a small number of repeats per question rather than formal error bars; we mitigate this by checking that conclusions hold across multiple models, benchmarks, and memory frameworks, but per-figure variance should be interpreted with this in mind.

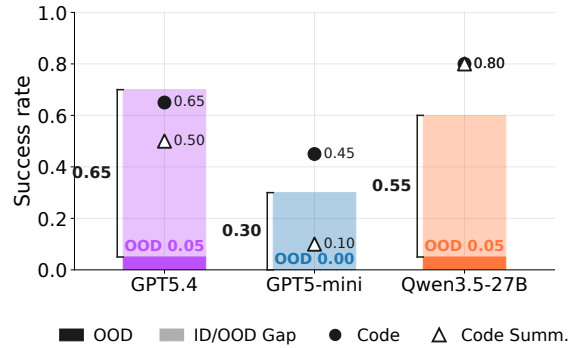


Figure 11: Memory turns overspecified and hurt performance outside of the compressed questions.

9. Broader Impacts

Our findings are diagnostic: they identify a failure mode in a widely adopted paradigm for agent memory and suggest more conservative default designs that preserve raw evidence and gate abstraction. The positive impact is to reduce silent regressions in deployed long-horizon agents and to direct research toward more reliable abstraction mechanisms. Negative impacts are indirect: better long-horizon agent memory could amplify the reach of autonomous systems whose downstream uses (persuasion, automation of consequential decisions) carry their own risks, and our diagnosis of failure modes does not constitute a fix. We do not release new datasets or models that pose specific misuse concerns, and the benchmarks we use are public.

References

- Frederic C. Bartlett. *Remembering: A Study in Experimental and Social Psychology*. Cambridge University Press, Cambridge, UK, 1932.
- Charles J. Brainerd and Valerie F. Reyna. Fuzzy-trace theory and false memory. *Current Directions in Psychological Science*, 11(5):164–169, 2002. doi: 10.1111/1467-8721.00192.
- Lee R. Brooks. Nonanalytic concept formation and memory for instances. In Eleanor Rosch and Barbara B. Lloyd, editors, *Cognition and Categorization*, pages 169–211. Lawrence Erlbaum Associates, Hillsdale, NJ, 1978.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. 2023. URL <https://arxiv.org/abs/2306.06070>.
- Yadin Dudai, Avi Karni, and Jan Born. The consolidation and transformation of memory. *Neuron*, 88(1):20–32, 2015. doi: 10.1016/j.neuron.2015.09.004.
- John H. Flavell. Metacognition and cognitive monitoring: A new area of cognitive–developmental inquiry. *American Psychologist*, 34(10):906–911, 1979. doi: 10.1037/0003-066X.34.10.906.
- Robert M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.
- Douglas L. Hintzman. “schema abstraction” in a multiple-trace memory model. *Psychological Review*, 93(4):411–428, 1986.
- Asher Koriati. Monitoring one’s own knowledge during study: A cue-utilization approach to judgments of learning. *Journal of Experimental Psychology: General*, 126(4):349–370, 1997. doi: 10.1037/0096-3445.126.4.349.
- Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016. doi: 10.1016/j.tics.2016.05.004.
- LangChain AI. LangMem: Long-term memory for language agents, 2024. URL <https://langchain-ai.github.io/langmem/>. Software documentation.
- Letta. Letta: Stateful agents framework, 2024. URL <https://www.letta.com>. Software documentation.
- Bingxuan Li, Jeonghwan Kim, Cheng Qian, Xiushi Chen, Eitan Anzenberg, Niran Kundapur, and Heng Ji. Pearl: Self-evolving assistant for time management with reinforcement learning. *arXiv preprint arXiv:2601.11957*, 2026.
- Gordon D. Logan. Toward an instance theory of automatization. *Psychological Review*, 95(4):492–527, 1988.

- Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Peter Jansen, Oyvind Tafjord, Niket Tandon, Li Zhang, Chris Callison-Burch, and Peter Clark. CLIN: A continually learning language agent for rapid task adaptation and generalization, 2023.
- James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995. doi: 10.1037/0033-295X.102.3.419.
- Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In G. H. Bower, editor, *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.
- Douglas L. Medin and Marguerite M. Schaffer. Context theory of classification learning. *Psychological Review*, 85(3): 207–238, 1978.
- Michael D. Moffitt. Arc-gen: A mimetic procedural benchmark generator for the abstraction and reasoning corpus, 2025. URL <https://arxiv.org/abs/2511.00162>.
- Morris Moscovitch, Roberto Cabeza, Gordon Winocur, and Lynn Nadel. Episodic memory and beyond: The hippocampus and neocortex in transformation. *Annual Review of Psychology*, 67:105–134, 2016.
- Arseny Moskvichev, Victor Vikram Odouard, and Melanie Mitchell. The conceptarc benchmark: Evaluating understanding and generalization in the arc domain, 2023. URL <https://arxiv.org/abs/2305.07141>.
- Thomas O. Nelson and Louis Narens. Metamemory: A theoretical framework and new findings. In G. H. Bower, editor, *The Psychology of Learning and Motivation*, volume 26, pages 125–173. Academic Press, 1990.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=jL7fwchScm>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.
- Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, New York, NY, USA, 2023. Association for Computing Machinery. doi: 10.1145/3586183.3606763.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Larry R Squire, Lisa Genzel, John T Wixted, and Richard G Morris. Memory consolidation. *Cold Spring Harbor perspectives in biology*, 7(8):a021766, 2015.
- Theodore Summers, Shunyu Yao, Karthik R Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=1i6ZCvflQJ>.
- Weinan Sun, Madhu Advani, Nelson Spruston, Andrew Saxe, and James E. Fitzgerald. Organizing memories for generalization in complementary learning systems. *Nature Neuroscience*, 26(8):1438–1448, 2023. doi: 10.1038/s41593-023-01382-9.
- Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In Vera Demberg, Kentaro Inui, and Lluís Marquez, editors, *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106, Rabat, Morocco, March 2026. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.333. URL <https://aclanthology.org/2026.eacl-long.333/>.

- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjana Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. 2024. URL <https://arxiv.org/abs/2407.18901>.
- Dorothy Tse, Rosamund F. Langston, Masaki Kakeyama, Ingrid Bethus, Patrick A. Spooner, Emma R. Wood, Menno P. Witter, and Richard G. M. Morris. Schemas and memory consolidation. *Science*, 316(5821):76–82, 2007. doi: 10.1126/science.1135935.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=FiMOM8gcct>.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. 2022. URL <https://arxiv.org/abs/2207.01206>.
- Jeffrey M. Zacks, Nicole K. Speer, Khena M. Swallow, Todd Samuel Braver, and Jeremy R. Reynolds. Event perception: a mind-brain perspective. *Psychological bulletin*, 133 2:273–93, 2007. URL <https://api.semanticscholar.org/CorpusID:10494362>.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. 2025.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19632–19642, 2024. doi: 10.1609/aaai.v38i17.29936.

Trajectory Logs (Summarized)												
Backbone	NoMem	All	64	32	16	Incr. Keep	One-Pass Keep	ACE _{GT}	ACE	AWM	DC	
GPT-5.4-Mini	18	47	44	49	51	40	41	—	00*	23 (−)	03*	
GPT-5-Mini	39	53	41	63	49	49	50	—	—	59 (−)	—	
Claude-Haiku-4.5	19	59	63	55	49	54	54	—	—	32 (−)	—	

Table 3: **Trajectory Log as a Strong Baseline (WebShop)**. Values are mean reward $\times 100$. **NoMem** (no-memory baseline); **Trajectory Logs (Summarized)** groups append-only summarized rollouts retrieved as in-context exemplars—**All/64/32/16** use the full pool or downsample to that many shots, while **Incr. Keep** and **One-Pass Keep** are LLM-curated retain/delete variants (streamed and one-pass, respectively); **ACE_{GT}** (ACE with ground-truth labels), **ACE** (ACE without GT), **AWM** (Agent Workflow Memory), **DC** (Dynamic Cheatsheet). A red (−) marks any non-trajectory-log approach that is beaten by at least one trajectory-log baseline in the same row. Best per row in **bold**. Cells marked — are not measured for that backbone. The results with * are evaluated by model GPT-5-Mini. Then we observed that its performance were near to 0, so we did not do remaining ACE or DC experiments on webshop.

	$ B =1$		$ B =8$	
	Auto	Force	Auto	Force
Avg. Covered	1.33	1.67	2.17	5.00
Avg. Fused	1.00	1.04	1.06	1.20
Buffer Size	3.33	—	50.00	—

Table 4: **Memory Coverage**.

Contents

A	Family and skill taxonomy [ARC-AGI]	17
A.1	Definitions	17
A.2	Family catalog	17
A.3	Skill catalog	19
B	Prompts [ARC-AGI]	21
B.1	Solver prompt	21
B.2	Tool helper bodies	26
B.3	Consolidator decision prompt	29
B.4	Consolidator extraction prompt	31
B.5	Failure recording prompt	33
B.6	Alternative schema for the consolidator extraction prompt	34
B.7	Strategy selection prompt	36

C	Strategy memory evolution over time [ARC-AGI]	37
C.1	How a strategy is traced across compress steps	37
C.2	Worked lineage example	37
D	Examples of wrong / low-quality strategies [ARC-AGI]	38
D.1	Vacuous abstraction	38
D.2	Concrete-vocabulary erosion	39
D.3	Foreign-family injection	39
D.4	Phantom strategies from failed attempts	40
D.5	Single-strategy collapse on multi-family compression	40
E	Examples of useful / well-formed strategies [ARC-AGI]	44
F	Additional Experiment Details	45
F.1	ALFWorld	45
F.2	WebShop	46
F.3	ScienceWorld	47
F.4	AppWorld	48
G	Case studies of memory failure modes	50
G.1	Erosion: good memory degrading over time	50
G.2	Collapsed memory: single-stage merge to one item	53
G.3	Useless memory items	54
H	WebShop	56
H.1	Memory artifacts from prior methods	56
I	Memory surgery	59
I.1	Workflow 8 ablation on WebShop	59
J	ScienceWorld	62
J.1	Abnormal memories	62
K	Additional Analyses	68
K.1	Auto-Mode Component Ablation	68
K.2	Memory Composition under Task Switching	68

A. Family and skill taxonomy [ARC-AGI]

This section defines the two terms “family” and “skill” and lists the six families and seven skills used in the ARC-AGI experiments.

A.1. Definitions

Our task generator decomposes an ARC-style task along two independent axes:

- **Family:** a rule that selects the subset of connected non-background objects in the input grid that participates in the task’s transformation.
- **Skill:** a fixed transformation applied to each selected object.

The two axes are orthogonal: the same family can be paired with different skills, and the same skill can be paired with different families. Some families are adapted from concept groups in ConceptARC; for example, `detect_inside_frame_relation` corresponds to the `INSIDEOUTSIDE` concept group.

A.2. Family catalog

The six families span five distinct selection criteria: two are based on intrinsic per-object attributes (color, size extremum), one on a global conditional cue (a marker cell that triggers a branch), one on a cross-object relation (shape-frequency mode), one on a spatial containment relation (objects strictly inside a frame), and one on a cross-grid structural property (a sub-scene of a two-panel input). The chosen skill is then applied uniformly to whichever objects the family selects.

Table 5: The six families evaluated in this paper, grouped by what they select on. The rightmost column gives the rule by which each family picks the object subset that participates in the task’s transformation. The first column reproduces the `strategy_id` field used to identify each family in the released code and data.

strategy_id	Selects by	Definition
<code>detect_color_property</code>	intrinsic (color)	objects of a target color (inferred from the examples)
<code>detect_largest_objects_select</code>	intrinsic (size extremum)	the largest object(s) by cell count (ties all selected)
<code>detect_key_marker_rule</code>	global conditional cue	all-or-none global selection gated by the upper-left cell’s color
<code>group_by_shape_then_select</code>	cross-object relation	objects in the shape-frequency mode group
<code>detect_inside_frame_relation</code>	spatial containment	objects strictly inside a hollow frame
<code>compose_horizontal</code>	cross-grid structure	the left or right sub-scene of a two-panel input

Figure 12 shows one task per family under a fixed skill of `recolor`: recolored objects in the output identify the family’s selection by color contrast.

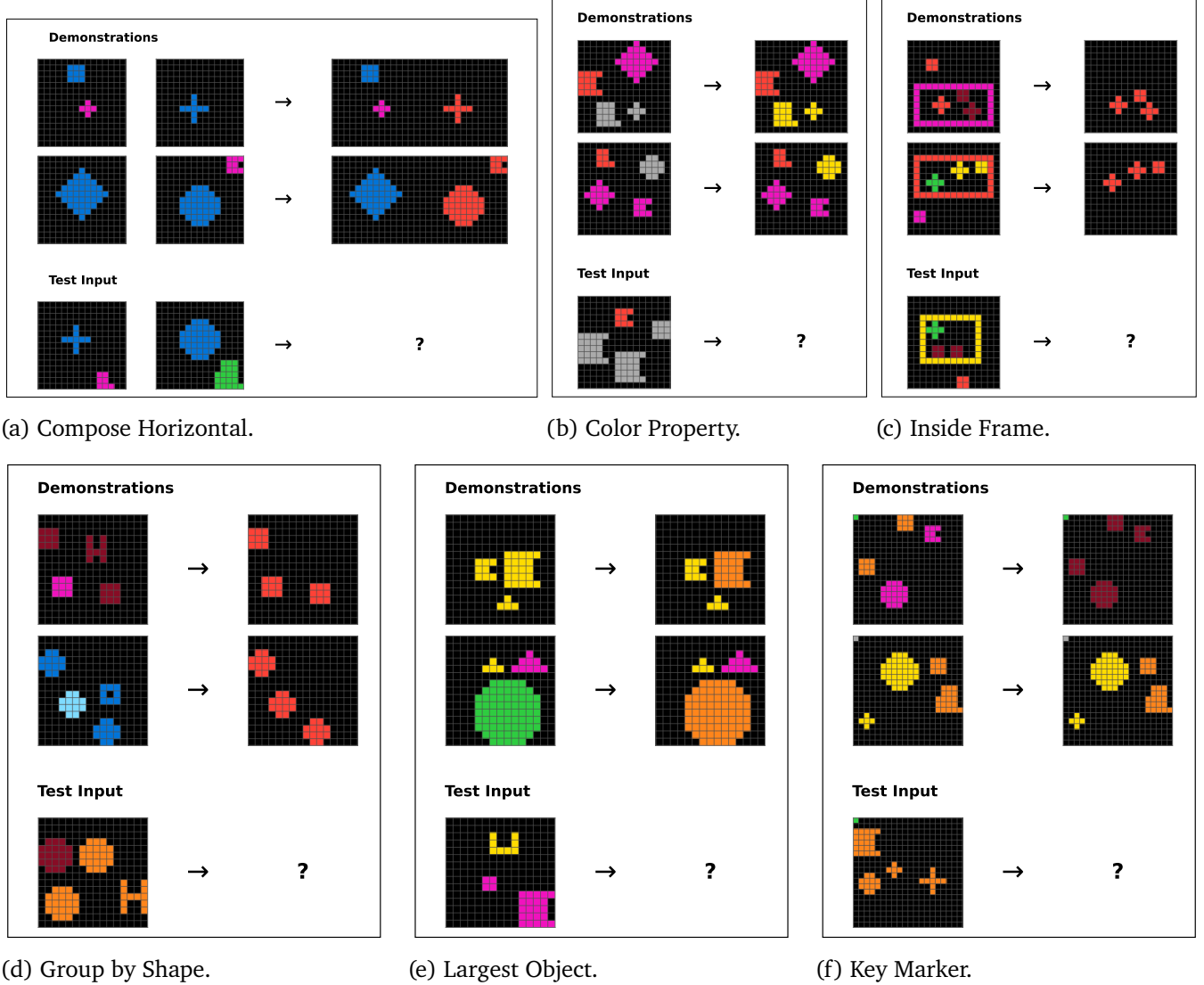


Figure 12: One task per family, all rendered with the skill axis fixed to recolor. Each panel shows two demonstration input–output pairs and a held-out test input. Recolored objects in the output identify which objects the family selects. Compose Horizontal takes two input grids per example and transforms the left sub-scene; the other five families operate on a single input. The Key Marker panel additionally illustrates conditional branching: when the upper-left marker color matches the trigger value, objects are recolored; otherwise the output equals the input. Cell colors follow the 10-color ARC palette; color 0 (black) denotes background and colors 1–9 denote object pixels.

A.3. Skill catalog

Each skill applies a fixed transformation to every object in the family’s selection.

Table 6: The seven skills evaluated in this paper. Each skill is applied uniformly to every object selected by the family.

skill_id	Definition
keep	Retain selected objects unchanged; erase non-selected objects to background.
border	Draw a one-cell border around each selected object.
recolor	Repaint each selected object in a single new color.
translate	Shift each selected object by a fixed offset.
flip_horizontal	Mirror each selected object within its bounding box.
mark_center	Mark the centroid cell of each selected object with a marker color.
hollow	Erase the interior of each selected object, leaving only its outer border.

Figure 13 shows the per-skill effect under a fixed family of `detect_color_property`, where the selection is visually simple (objects whose color matches a target predicate demonstrated by the examples), isolating the skill’s per-object action.

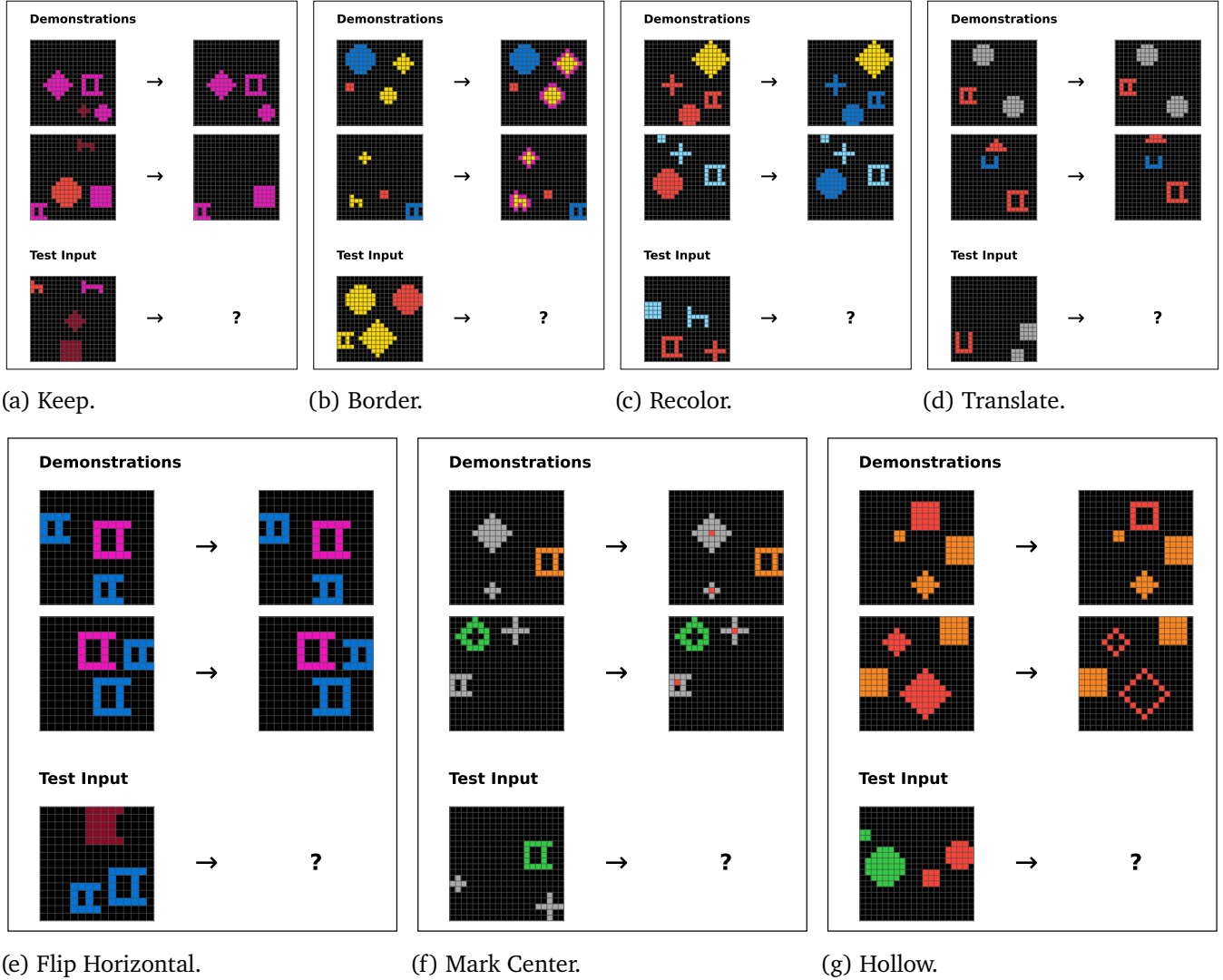


Figure 13: One task per skill, all rendered with the family axis fixed to `detect_color_property`. Each panel shows two demonstration input–output pairs and a held-out test input. The family selects objects whose color matches a target property demonstrated by the examples; the skill then applies its transformation to those objects only.

B. Prompts [ARC-AGI]

The experiments use three prompts: a solver prompt issued once per task, a consolidator decision prompt that selects an action on the recent history buffer after each passing step, and a consolidator extraction prompt that produces a replacement strategy memory from a selected subset of history. To make every field concrete we reproduce one full step from a real run in which the current solve passes; the history buffer carries two prior passing trajectories and strategy memory holds one entry distilled at an earlier step. Each prompt is shown together with the response it elicited. Where long content repeats across prompts (the eight extra IO examples for the current task; the 8 tool-buffer helper bodies; grids and solve code already shown above), we mark the elision in square brackets; everything else is byte-for-byte from the run logs.

B.1. Solver prompt

The solver prompt is issued once per task at the start of the step. It bundles the 10 IO examples for the current task, the existing strategy memory, the tool-buffer helper signatures, and the recent history buffer into one call.

Input.

Solver prompt (Examples 3–10 and helper bodies elided)

```
You are an expert Python programmer.
**Current Task:**
Write a Python function `solve` that passes the following examples.
Use the signature `def solve(grid):`.
Example 1:
Input:
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 2 2 2 2 2 2 2 2 2 2 0 0 0 0
0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 2 0 0 0
0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 2 0 0 0
0 0 0 0 0 0 2 7 7 0 0 0 0 0 7 0 2 0 0 0
0 0 0 0 0 0 2 7 7 0 0 0 0 0 7 0 2 0 0 0
0 0 0 0 0 0 2 0 0 0 0 0 0 0 7 7 2 0 0 0
0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 2 0 0 0
0 0 0 0 0 0 2 0 0 5 5 5 0 0 0 0 2 0 0 0
0 0 0 0 0 0 2 0 0 0 5 0 0 0 0 0 2 0 0 0
0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 2 0 0 0
0 0 0 3 3 0 2 0 0 0 0 0 0 0 0 0 2 0 0 0
0 0 0 3 3 0 2 0 0 0 0 0 0 0 0 0 2 0 0 0
0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 2 0 0 0
0 0 0 0 0 0 2 2 2 2 2 2 2 2 2 2 2 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Output:
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 7 7 0 0 0 0 0 7 0 0 0 0 0
0 0 0 0 0 0 0 7 7 0 0 0 0 0 7 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 7 7 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 5 5 5 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 5 0 0 0 0 0 0 0 0 0 0
```

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

Example 2:

Input:

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 4 4 4 4 4 4 4 4 4 4 4 0 0 0 0 0
0 0 0 0 4 0 0 0 0 0 0 0 0 0 4 0 0 0 0 0
0 0 0 0 4 0 0 0 0 6 6 6 0 0 4 0 0 0 0 0
0 0 0 0 4 0 0 0 0 6 0 0 0 4 0 0 0 0 0 0
0 0 0 0 4 0 0 0 0 0 0 0 0 4 0 0 0 0 0 0
0 0 0 0 4 0 0 0 0 0 0 0 0 4 0 0 0 0 0 0
0 0 0 0 4 5 0 0 0 0 0 0 0 4 0 0 0 0 0 0
0 0 0 0 4 5 0 0 0 5 0 0 0 4 0 0 0 0 0 0
0 0 0 0 4 5 5 0 0 5 0 0 0 4 0 0 0 0 0 0
0 0 0 0 4 0 0 0 0 5 5 0 0 4 0 0 0 0 0 0
0 0 0 0 4 0 0 0 0 0 0 0 0 4 0 0 0 0 0 0
0 0 0 0 4 4 4 4 4 4 4 4 4 4 4 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 7 7 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 7 7 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

Output:

```
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 6 6 6 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 6 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 5 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 5 0 0 0 5 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 5 5 0 0 5 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 5 5 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

[Examples 3 through 10 elided -- eight more 20x20 input/output grid pairs from this task, same Input :/Output: format.]

****Memory**** (all extracted patterns -- use as reference; apply what is relevant):

[1]

When to use: The task has two same-sized input grids and the output has the same height but double the width, arranged as a left-right concatenation. The left half reproduces the shape pattern from the first input but normalizes every non-background cell to one single fill color, while the right half copies the second input unchanged.

Strategy: (1) Verify that the two inputs have identical dimensions and that the output width should be the sum of their widths. (2) Build the left half from the first input by keeping background

cells as background and changing every non-background cell, regardless of its original color, to the common fill color required by the task. This preserves the exact geometry and positions of all shapes from the first input but removes color distinctions. (3) Build the right half by copying the second input exactly without moving or recoloring anything. (4) Concatenate the transformed first grid and the unchanged second grid horizontally, row by row, to form the final output.

[Tool Buffer - Callable Helpers]

These helper functions are callable in the execution environment and should be reused by direct call when applicable.

Callable helper names: apply_border, apply_flip_horizontal, apply_hollow, apply_mark_center, apply_op_per_object, apply_recolor, apply_translate, extract_objects

****DO NOT** redefine these helpers.** Do not use globals(), locals(), or dynamic lookup; call helpers by name only.

[Helper bodies elided here -- the 8 inlined function definitions are reproduced in full in the Tool helper definitions subsection that follows. The solver may call them but must not redefine them.]

[History Buffer - Previous Task Trajectories - Reference Only]

These are previous task-specific trajectories. They are for pattern reference only and are NOT callable.

[Task A]

Input:

```
5 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 8 8 8 8 0 0 0 0 0 0 0 0 0 0
0 8 8 8 8 0 3 3 3 3 0 0 0 0 0
0 8 8 8 8 0 3 0 3 0 0 0 0 0 0
0 8 8 8 8 0 3 0 3 0 0 0 0 0 0
0 0 0 0 0 0 3 3 3 3 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 8 0 0 0 0
0 0 0 0 0 0 0 0 0 0 8 0 0 0 0
0 0 0 0 0 0 0 0 0 0 8 8 8 8 0
0 0 0 0 0 0 8 0 0 0 8 0 0 8 0
0 0 0 0 0 0 8 8 8 0 0 0 0 0 0
0 0 0 0 0 0 8 0 8 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

Output:

```
5 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

Solution:

```
def solve_task_A(grid):
    h = len(grid)
    w = len(grid[0]) if h else 0

    # Control is the top-left cell.
    marker = grid[0][0] if h and w else 0

    # From the examples:
```

```
# - marker 4 => keep grid unchanged
# - marker 5 => clear everything except the marker cell
if marker == 4:
    return [row[:] for row in grid]

out = [[0] * w for _ in range(h)]
if h and w:
    out[0][0] = marker
return out

---

[Task B]
Input:
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 2 2 0 0 0 0 0 0 0 0 0 0
0 0 0 2 2 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 6 6 6 6 6 6 6 6 6 6 6 6 6 0
0 6 0 0 0 0 0 0 9 9 0 0 0 6 0
0 6 0 0 0 2 0 0 9 9 0 0 0 6 0
0 6 0 0 2 2 2 0 0 9 0 0 6 0
0 6 0 0 0 2 0 0 9 9 9 0 6 0
0 6 0 0 0 0 0 0 0 9 0 0 6 0
0 6 6 6 6 6 6 6 6 6 6 6 6 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Output:
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 2 2 0 0 0 0 0
0 0 0 0 0 2 0 0 2 2 0 0 0 0 0
0 0 0 0 2 2 2 0 0 0 2 0 0 0 0
0 0 0 0 0 2 0 0 0 2 2 2 0 0 0
0 0 0 0 0 0 0 0 0 0 2 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Solution:

def solve_task_B(grid):
    h = len(grid)
    w = len(grid[0]) if h else 0

    # Find the largest hollow rectangular frame:
    # a color whose bbox perimeter is fully occupied by that color.
    objs = extract_objects(grid)
    best = None

    for obj in objs:
        cells = set(obj["cells"])
        b = obj["bbox"]
        top, left, bottom, right = b["top"], b["left"], b["bottom"], b["right"]

        # Need a non-degenerate rectangle
        if bottom - top < 2 or right - left < 2:
            continue
```

```
ok = True
# Check full perimeter
for c in range(left, right + 1):
    if (top, c) not in cells or (bottom, c) not in cells:
        ok = False
        break
if ok:
    for r in range(top, bottom + 1):
        if (r, left) not in cells or (r, right) not in cells:
            ok = False
            break
if ok:
    area = (bottom - top + 1) * (right - left + 1)
    if best is None or area > best[0]:
        best = (area, obj)

out = [[0] * w for _ in range(h)]
if best is None:
    return out

frame = best[1]
b = frame["bbox"]
top, left, bottom, right = b["top"], b["left"], b["bottom"], b["right"]
frame_cells = set(frame["cells"])

# Keep only cells strictly inside the frame, excluding the frame itself,
# and recolor everything kept to 2.
for r in range(top + 1, bottom):
    for c in range(left + 1, right):
        if (r, c) not in frame_cells and grid[r][c] != 0:
            out[r][c] = 2

return out
Reason briefly from the examples, then write the simplest correct `solve`.
Do not output long chain-of-thought or extra prose.
```

Response. The model's reply, extracted as the first Markdown-fenced Python block and graded as passing on all 10 examples:

Solver response (graded passing)

```
def solve(grid):
    h = len(grid)
    w = len(grid[0]) if h else 0

    # Find the largest rectangular hollow frame:
    # a single-color object whose entire bbox perimeter is filled by that color.
    def largest_frame():
        objs = extract_objects(grid)
        best = None # (area, obj)
        for obj in objs:
            cells = set(obj["cells"])
            b = obj["bbox"]
            top, left, bottom, right = b["top"], b["left"], b["bottom"], b["right"]

            # Need a nontrivial hollow rectangle
            if bottom - top < 2 or right - left < 2:
                continue
```

```
    ok = True
    for c in range(left, right + 1):
        if (top, c) not in cells or (bottom, c) not in cells:
            ok = False
            break
    if ok:
        for r in range(top, bottom + 1):
            if (r, left) not in cells or (r, right) not in cells:
                ok = False
                break

    if ok:
        area = (bottom - top + 1) * (right - left + 1)
        if best is None or area > best[0]:
            best = (area, obj)
    return best[1] if best else None

frame = largest_frame()
out = [[0] * w for _ in range(h)]
if frame is None:
    return out

b = frame["bbox"]
top, left, bottom, right = b["top"], b["left"], b["bottom"], b["right"]

# Keep only non-frame cells strictly inside the frame; remove the frame and everything outside.
for r in range(top + 1, bottom):
    for c in range(left + 1, right):
        if grid[r][c] != 0:
            out[r][c] = grid[r][c]

return out
```

B.2. Tool helper bodies

Tool helpers are defined per skill (section A); they are used both to generate the ground-truth solver code for each task and to support the LLM solver at inference, inlined into the Tool Buffer block of section B.1 and section B.7. The full set is reproduced below.

Tool Buffer (verbatim expansion of the elision in section B.1 and section B.7)

```
[Tool Buffer - Callable Helpers]
These helper functions are callable in the execution environment and should be reused by direct call
when applicable.
Callable helper names: apply_border, apply_flip_horizontal, apply_hollow, apply_mark_center,
    apply_op_per_object, apply_recolor, apply_translate, extract_objects
**DO NOT redefine these helpers.** Do not use globals(), locals(), or dynamic lookup; call helpers by
    name only.

def apply_border(
    grid: List[List[int]],
    obj: Dict[str, Any],
    border_color: int,
) -> List[List[int]]:
    """Paint ``border_color`` at each background 4-neighbour of ``obj``. Obj cells unchanged."""
    cells = obj.get("cells") or []
    if not grid or not grid[0] or not cells:
        return grid
    h, w = len(grid), len(grid[0])
    obj_set = set(cells)
    bc = int(border_color)
```

```
for r, c in cells:
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < h and 0 <= nc < w and grid[nr][nc] == 0 and (nr, nc) not in obj_set:
            grid[nr][nc] = bc
return grid

def apply_flip_horizontal(grid: List[List[int]], obj: Dict[str, Any]) -> List[List[int]]:
    """Mirror ``obj`` left/right around its bbox center column."""
    cells = obj.get("cells") or []
    if not grid or not grid[0] or not cells:
        return grid
    h, w = len(grid), len(grid[0])
    color = int(obj.get("color", 0))
    bbox = obj["bbox"]
    cc = (bbox["left"] + bbox["right"]) / 2.0
    for r, c in cells:
        if 0 <= r < h and 0 <= c < w and grid[r][c] == color:
            grid[r][c] = 0
    for r, c in cells:
        nc = int(cc - (c - cc))
        if 0 <= r < h and 0 <= nc < w:
            grid[r][nc] = color
    return grid

def apply_hollow(
    grid: List[List[int]],
    obj: Dict[str, Any],
    fill_color: int = 0,
) -> List[List[int]]:
    """Keep ``obj`` cells whose 4-neighbour is bg or off-grid (border); set interior cells to ``fill_color``. Foreign-obj neighbours do NOT count as border."""
    cells = obj.get("cells") or []
    if not grid or not grid[0] or not cells:
        return grid
    h, w = len(grid), len(grid[0])
    obj_set = set(cells)
    boundary: set = set()
    for r, c in cells:
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if not (0 <= nr < h and 0 <= nc < w):
                boundary.add((r, c))
                break
            if (nr, nc) not in obj_set and grid[nr][nc] == 0:
                boundary.add((r, c))
                break
    fc = int(fill_color)
    for r, c in cells:
        if (r, c) not in boundary:
            grid[r][c] = fc
    return grid

def apply_mark_center(
    grid: List[List[int]],
    obj: Dict[str, Any],
    mark_color: int = 0,
) -> List[List[int]]:
    """Write ``mark_color`` at ``obj``'s integer bbox center cell.
    """
    cells = obj.get("cells") or []
    if not grid or not grid[0] or not cells:
        return grid
    h, w = len(grid), len(grid[0])
    bbox = obj["bbox"]
```

```
cr = (bbox["top"] + bbox["bottom"]) // 2
cc = (bbox["left"] + bbox["right"]) // 2
target = int(mark_color)
if target <= 0:
    color = int(obj.get("color", 0))
    target = (color
    if target == color:
        target = ((color + 1)
if 0 <= cr < h and 0 <= cc < w:
    grid[cr][cc] = target
return grid

def apply_op_per_object(
    grid: List[List[int]],
    op_name: str,
    **params: Any,
) -> List[List[int]]:
    """Apply ``op_name`` to every connected component INDEPENDENTLY (on its own isolated patch), then
    OR-composite the transformed patches onto a fresh blank canvas. Returns the new canvas."""
    per_object_fns = {
        "recolor": apply_recolor,
        "translate": apply_translate,
        "flip_horizontal": apply_flip_horizontal,
        "border": apply_border,
        "hollow": apply_hollow,
        "mark_center": apply_mark_center,
        "center_mark": apply_mark_center,
    }
    if not grid or not grid[0]:
        return [list(row) for row in grid]
    h, w = len(grid), len(grid[0])
    fn = per_object_fns.get(op_name)
    if fn is None:
        return [list(row) for row in grid]
    out = [[0] * w for _ in range(h)]
    for obj in extract_objects(grid):
        if not obj.get("cells"):
            continue
        patch = [[0] * w for _ in range(h)]
        for r, c in obj["cells"]:
            patch[r][c] = obj["color"]
        patch_objs = extract_objects(patch)
        if patch_objs:
            fn(patch, patch_objs[0], **params)
        for r in range(h):
            for c in range(w):
                if patch[r][c]:
                    out[r][c] = patch[r][c]
    return out

def apply_recolor(
    grid: List[List[int]],
    obj: Dict[str, Any],
    new_color: int,
) -> List[List[int]]:
    """Recolor all ``obj`` cells to ``new_color``.
    """
    cells = obj.get("cells") or []
    if not grid or not grid[0] or not cells:
        return grid
    h, w = len(grid), len(grid[0])
    target = int(new_color)
    for r, c in cells:
        if 0 <= r < h and 0 <= c < w:
            grid[r][c] = target
```

```
    return grid

def apply_translate(
    grid: List[List[int]],
    obj: Dict[str, Any],
    dr: int,
    dc: int,
) -> List[List[int]]:
    """Shift ``obj`` cells by ``(dr, dc)``; off-grid cells are dropped. Cells that still hold ``obj.
    color`` at their original position are cleared first so prior per-object writes aren't stomped."""
    cells = obj.get("cells") or []
    if not grid or not grid[0] or not cells:
        return grid
    h, w = len(grid), len(grid[0])
    color = int(obj.get("color", 0))
    for r, c in cells:
        if 0 <= r < h and 0 <= c < w and grid[r][c] == color:
            grid[r][c] = 0
    for r, c in cells:
        nr, nc = r + int(dr), c + int(dc)
        if 0 <= nr < h and 0 <= nc < w:
            grid[nr][nc] = color
    return grid

def extract_objects(grid: List[List[int]], background: int = 0) -> List[Dict[str, Any]]:
    """Return 4-connected non-background components as dicts ``{cells, color, bbox, size}``, in row-
    major scan order."""
    if not grid or not grid[0]:
        return []
    h, w = len(grid), len(grid[0])
    seen = [[False] * w for _ in range(h)]
    objects: List[Dict[str, Any]] = []
    for sr in range(h):
        for sc in range(w):
            if seen[sr][sc] or grid[sr][sc] == background:
                continue
            color = grid[sr][sc]
            stack: List[Tuple[int, int]] = [(sr, sc)]
            cells: List[Tuple[int, int]] = []
            while stack:
                r, c = stack.pop()
                if not (0 <= r < h and 0 <= c < w) or seen[r][c] or grid[r][c] != color:
                    continue
                seen[r][c] = True
                cells.append((r, c))
                stack.extend([(r - 1, c), (r + 1, c), (r, c - 1), (r, c + 1)])
            if not cells:
                continue
            top = min(r for r, _ in cells); bottom = max(r for r, _ in cells)
            left = min(c for _, c in cells); right = max(c for _, c in cells)
            objects.append({
                "cells": cells,
                "color": int(color),
                "bbox": {"top": top, "left": left, "bottom": bottom, "right": right},
                "size": len(cells),
            })
    return objects
```

B.3. Consolidator decision prompt

After the solver's response is graded passing, the consolidator decision prompt is issued. The history buffer at this point contains three entries: the two carryover trajectories shown in the solver's history block above, plus the trajectory just

produced for the current task (one IO pair plus the response above).

Input.

Consolidator decision prompt (history bodies elided)

```
You manage a History buffer (recent LLM solve traces) and a Strategy memory
(distilled patterns). Pick ONE action given the state below
(1 new this step, 2 carryover).
```

```
## History buffer (3 entries; indices: 1..2 = carryover, 3..3 = new this step; capacity=50, FIFO;
 1 IO + solve code per entry)
```

```
[History 1 -- carryover -- task_id=A -- passed]
Example IO pair:
[Same 15x15 IO pair as solver's [Task A] block above.]
Solve code:
```python
[Same as solve_task_A body above.]
```

---
```

```
[History 2 -- carryover -- task_id=B -- passed]
Example IO pair:
[Same 15x15 IO pair as solver's [Task B] block above.]
Solve code:
```python
[Same as solve_task_B body above.]
```

---
```

```
[History 3 -- new this step -- task_id=C -- passed]
Example IO pair:
[Same 20x20 IO pair as solver's Example 1 above.]
Solve code:
```python
[Same as the solver response above.]
```
```

```
## Strategy memory (1 entries; capacity=unbounded)
```

```
- Strategy 1: When to use: The task has two same-sized input grids and the output has the same
height but double the width, arranged as a left-right concatenation. The left half reproduces the
shape pattern from the first input but normalizes every non-background cell to one single fill
color, while the right half copies the second input unchanged.
```

```
Strategy: (1) Verify that the two inputs have identical dimensions and that the output width should
be the sum of their widths. (2) Build the left half from the first input by keeping background
cells as background and changing every non-background cell, regardless of its original color, to
the common fill color required by the task. This preserves the exact geometry and positions of
all shapes from the first input but removes color distinctions. (3) Build the right half by
copying the second input exactly without moving or recoloring anything. (4) Concatenate the
transformed first grid and the unchanged second grid horizontally, row by row, to form the final
output.
```

```
Actions:
- Keep: leave history and memory unchanged.
- Remove: drop history entries; "fn_indices" required (>=1, 1..H).
- Strategy extraction: extract a strategy memory entry from selected history; "fn_indices"
required (>=1, 1..H; pick >=2 when entries share a plan). Selected entries are CONSUMED from
history; their information moves into strategy memory.
```

```
Return JSON:
{"action": "Keep" | "Remove" | "Strategy extraction", "reason": "<brief>", "fn_indices": [...]}
(omit "fn_indices" for Keep; required otherwise)
```

Response. The model selects `Strategy extraction` on history entries 2 and 3, the two `detect_inside_frame_relation` trajectories that share a plan:

Decision response

```
{"action": "Strategy extraction", "reason": "Histories 2 and 3 share the same reusable plan: detect the largest hollow rectangular frame, remove the frame/outside, and keep only nonzero cells strictly inside (optionally recoloring depending on task). This is a strong recurring pattern worth storing.", "fn_indices": [2, 3]}
```

B.4. Consolidator extraction prompt

Because the decision was `Strategy extraction`, the extraction prompt is then issued on the subset of history selected by the decision step. The current strategy buffer is also supplied so the model can choose to retain, merge, or drop existing entries.

Input.

Consolidator extraction prompt (task bodies elided)

You are converting a batch of K solved ARC-AGI tasks into reusable natural-language memory entries.

You will see:

1. The current strategy buffer (1-based indices 1.. N). You may **RETAIN** entries by index, **MERGE** several into a cleaner entry, or **DROP** entries by omitting them from the output.
2. K input tasks (1-based indices 1.. K), each with description, 5 sample IO pairs, and reference solution code.

Goal: produce the **full replacement strategy buffer** as a JSON list of entries. Each entry is exactly one of:

- Retain unchanged:
`{"from_existing": [i, j, ...]}`
Lists ≥ 1 existing indices; each listed index becomes its own kept-as-is entry. NO other fields. Only valid when a '### Current strategy buffer' section appears above; do NOT use if buffer is empty.
- New (distilled from task solutions):
`{"when_to_use": "...", "solve_strategy": "...", "from_functions": [k1, k2, ...]}`
Must have BOTH text fields and ≥ 1 "from_functions" index into the K input tasks.
- Merge (existing entries + optionally task evidence into one cleaner entry):
`{"when_to_use": "...", "solve_strategy": "...", "from_existing": [i, ...], "from_functions": [k, ...]}`
Must have BOTH text fields; at least one of "from_existing" / "from_functions" non-empty.

Field definitions:

- "when_to_use": describe the visual/task patterns that should trigger this memory in a future ARC task. Focus on observable cues: output size, object movement, color changes, cropping, symmetry, counting, markers, repeated patterns, etc.

- "solve_strategy": detailed step-by-step reusable solving strategy. Do NOT summarize a single task; generalize the solution into an abstract procedure that could be applied to similar tasks.

Content rules (apply to every new/merge entry):

- Do not mention task-specific colors as fixed colors. Replace them with semantic roles such as background color, target color, marker color, object color, fill color, etc.
- Do not mention task-specific coordinates unless they express a reusable relation (top-left, center, border, same row, same column, inside, adjacent, symmetric position).
- Be concrete and operational.
- The strategy should be detailed enough that another model could apply it to a new ARC task.
- Avoid vague statements like "find the pattern" or "transform the object."
- If a pattern is too task-specific, still emit it, but make "when_to_use" narrow.

Schema / structural rules:

- One input task index may appear in multiple "from_functions" lists (one task can illustrate several patterns). Multiple input tasks may collapse into one entry (preferred when they share an algorithmic plan).
- Existing indices not referenced anywhere in your output are dropped from the new buffer.
- Do NOT re-emit existing entry text -- reference by index instead.
- You may output an empty list to drop everything.
- Output AT MOST N entries. Pick the most reusable, distinct patterns; merge near-duplicates rather than listing them separately.

Reply with a JSON list only. Example (mixing all three entry kinds):

```
[
  {"from_existing": [1, 4]},
  {"when_to_use": "Output size matches input; one object color is the most frequent non-bg ...",
   "solve_strategy": "(1) detect background, (2) ... (3) recolor",
   "from_functions": [2, 3]},
  {"when_to_use": "...",
   "solve_strategy": "merged plan that subsumes prior entry 2 and adds the diagonal axis case",
   "from_existing": [2], "from_functions": [5]}
]
```

Current strategy buffer (1-based indices):

1. When to use: The task has two same-sized input grids and the output has the same height but double the width, arranged as a left-right concatenation. The left half reproduces the shape pattern from the first input but normalizes every non-background cell to one single fill color, while the right half copies the second input unchanged.

Strategy: (1) Verify that the two inputs have identical dimensions and that the output width should be the sum of their widths. (2) Build the left half from the first input by keeping background cells as background and changing every non-background cell, regardless of its original color, to the common fill color required by the task. This preserves the exact geometry and positions of all shapes from the first input but removes color distinctions. (3) Build the right half by copying the second input exactly without moving or recoloring anything. (4) Concatenate the transformed first grid and the unchanged second grid horizontally, row by row, to form the final output.

Input tasks (1-based indices):

Task 1:

Example IO pair:

[Same 15x15 IO pair as solver's [Task B] block above.]

Solve code:

```
```python
[Same as solve_task_B body above.]
```
```

Outcome: passed

Task 2:

Example IO pair:

[Same 20x20 IO pair as solver's Example 1 above.]

```
Solve code:
```python
[Same as the solver response above.]
```
Outcome: passed
```

Response. The model drops the existing `compose_horizontal` entry and emits two new entries, both distilled from the inside-frame trajectories:

Extraction response

```
[
  {
    "when_to_use": "The input contains a large hollow rectangular border or frame made of one color, with other smaller objects both inside and outside it, and the output keeps the same grid size but removes the frame and everything outside it. Use this when the key distinction is whether cells lie strictly inside the largest rectangular enclosure.",
    "solve_strategy": "(1) Identify connected color objects and test which ones form a hollow axis-aligned rectangle: their bounding box must be at least 3x3, and every cell on that box's perimeter must belong to the object. (2) If multiple such frames exist, choose the largest by bounding-box area. (3) Create an all-background output grid of the same size. (4) Consider only cells strictly inside the chosen frame, excluding the frame border itself. (5) Copy the interior non-background cells to the same positions in the output, while leaving all cells on the frame and all cells outside the frame as background. This extracts the contents of the frame without moving them.",
    "from_functions": [2]
  },
  {
    "when_to_use": "The input contains a large hollow rectangular frame enclosing one or more objects, and the output keeps only the objects strictly inside that frame but normalizes their colors to a single target color. Use this when inside-vs-outside relation matters and the surviving interior shapes all become one uniform fill color.",
    "solve_strategy": "(1) Detect the largest hollow rectangular frame by finding a connected single-color object whose bounding box perimeter is completely occupied by that object's cells. Require a non-degenerate rectangle so there is a true interior. (2) Build an all-background output grid with the same dimensions as the input. (3) Scan only the cells strictly inside the frame, not including the border. (4) For every non-background cell found in that interior region, write a cell at the same location in the output using one common fill color, regardless of its original color. (5) Leave the frame itself, all exterior objects, and all background cells as background.",
    "from_functions": [1]
  }
]
```

B.5. Failure recording prompt

In some experimental conditions the history buffer also records steps whose solve was graded failing. Failed entries carry a Python-comment banner that prepends the failed solve code with one input/output triple from the failure: the input on which the candidate failed, the expected output, and what the candidate produced. Because the banner is comment-only, it is valid Python inside the same code fence the solver prompt's history block already uses. Below is one such entry from a real run; the input grids were 16x16, abbreviated to the first four rows for readability.

Failed-step entry inside the history buffer

```
[Task X]

Solution:
```

```
# [FAILED] This solution did not pass all evaluation examples.
# Wrong-IO sample (input / expected / got_or_error):
# [1] input:
# 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
# 0 0 0 0 0 0 0 0 0 0 0 0 6 0 0
# 0 0 0 0 0 0 0 0 0 0 0 0 6 0 0
# 0 0 0 0 0 0 0 0 0 0 6 6 6 6 0
# [...12 more rows elided...]
# expected:
# 0 0 0 0 0 0 0 0 0 0 0 8 0 0 0
# 0 0 0 0 0 0 0 0 0 0 8 6 8 0 0
# 0 0 0 0 0 0 0 0 0 8 8 6 8 8 0
# 0 0 0 0 0 0 0 8 6 6 6 6 6 8
# [...12 more rows elided...]
# got:
# 0 0 0 0 0 0 0 0 0 0 8 8 8 0 0
# 0 0 0 0 0 0 0 0 0 8 6 8 0 0
# 0 0 0 0 0 0 0 8 8 8 6 8 8 8
# 0 0 0 0 0 0 0 8 6 6 6 6 6 8
# [...12 more rows elided...]
# ---
def solve_task_X(grid):
    h, w = len(grid), len(grid[0])
    out = [row[:] for row in grid]

    for r in range(h):
        for c in range(w):
            if grid[r][c] != 0:
                continue
            for dr in (-1, 0, 1):
                for dc in (-1, 0, 1):
                    if dr == 0 and dc == 0:
                        continue
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < h and 0 <= nc < w and grid[nr][nc] != 0:
                        out[r][c] = 8

    return out
```

B.6. Alternative schema for the consolidator extraction prompt

An alternative schema for the consolidator extraction prompt asks the model to emit a flat JSON list of strategies, where each entry is one of three kinds: a retain (`from_existing`), a new pattern (`strategy` text plus `from_functions`), or a merge that combines existing entries with optional new evidence. Strategy text is a single free-form string rather than the structured (`when_to_use`, `solve_strategy`) pair used in the prompt above. We reproduce one full call from a real run; the buffer of existing strategies was empty at this step, so the model returns one new entry distilled from the single solve function shown.

Input.

Alternative extraction prompt

Identify the high-level algorithmic strategies used in these functions (numbered 1 to 1). Output the ****FULL replacement strategy buffer**** as a JSON list. Each output entry is one of:

- ****Retain**** existing strategies unchanged: ````{"from_existing": [i, j, ...]}```` -- list any number of 1-based existing indices (the buffer is currently empty). Each listed index becomes its own kept-as-is entry in the new buffer (compact form for keeping several entries; equivalent to writing one ````{"from_existing": [i]}```` per index). Omit "strategy" entirely.

```
- **New** pattern extracted from the functions: ``{"strategy": "<text>", "from_functions": [k1, k2, ...]}``  
  where ``k*`` are 1-based indices into the functions below.  
- **Merge** existing entries (and optionally new function evidence) into a single  
  cleaner description: ``{"strategy": "<merged text>", "from_existing": [i, j, ...], "from_functions":  
    [k, ...]}``.  
  Both index lists may have multiple entries; "from_functions" may be omitted if the merge is purely  
  existing-only.  
  
Hard rules:  
- Retain entries have NO "strategy" field and reference  $\geq 1$  existing index. Multiple  
  indices in one retain entry mean "keep all of them as separate entries".  
- New / merge entries (with "strategy") MUST cite at least one source via "from_existing" or "  
  from_functions".  
- The difference between retain (multi-index) and merge: retain produces N separate  
  entries unchanged; merge produces ONE new entry whose text is your "strategy" field.  
- Existing indices not referenced anywhere in your output are dropped.  
- Do NOT copy existing strategy text verbatim -- use the index instead.  
- You may output an empty list to drop everything.  
Functions are numbered 1..1. Reference them in "from_functions" when extracting a new pattern.  
  Existing strategies are referenced in "from_existing" by their 1-based index in the current  
  strategy buffer.  
  
### Function 1:  
``python  
def solve(grid):  
    objects = extract_objects(grid)  
    output = [list(row) for row in grid]  
    if not objects:  
        return output  
    _max_size = max(int(o.get('size', len(o.get('cells') or []))) for o in objects)  
    for obj in objects:  
        _obj_size = int(obj.get('size', len(obj.get('cells') or [])))  
        if _obj_size == _max_size:  
            pass  
        else:  
            _remove_color = int(obj.get('color', 0))  
            for _rr, _cc in (obj.get('cells') or []):  
                if 0 <= _rr < len(output) and 0 <= _cc < len(output[0]) and output[_rr][_cc] ==  
_remove_color:  
                    output[_rr][_cc] = 0  
    return output  
...```  
  
Reply with a JSON list only. Examples:  
[  
  {"from_existing": [1, 4, 7]},  
  {"strategy": "decompose grid -> process each part -> concatenate", "from_functions": [2, 4]},  
  {"strategy": "conditional branch on symmetry: existing rule extended to handle the new diagonal  
    axis seen in functions 3 and 5",  
    "from_existing": [2, 3], "from_functions": [3, 5]}  
]
```

Response.

Extraction response

```
[  
  {  
    "strategy": "Extract connected objects, identify the maximum-size object class (keeping all ties),  
      and erase every smaller object by setting its cells to background while leaving the largest  
      object(s) unchanged.",
```

```
    "from_functions": [1]
  }
]
```

B.7. Strategy selection prompt

An alternative invocation pattern for the solver decomposes the call into a strategy-selection step followed by a synthesis step. The selection prompt provides three inputs: the current callable helper buffer, half of the task's input-output examples, and the current strategy memory. The model replies with a JSON object that names which existing entry to apply; in the unconstrained variant it may instead propose a new one. The selected strategy text is then injected into the synthesis prompt's memory block in place of the full memory listing. We reproduce one full call from a real run; the helper-function bodies and four of the five input-output example pairs are elided.

Input.

Strategy selection prompt (helper bodies and all five IO examples elided)

```
Choose a strategy for this task. Below are, in order: (1) the tool buffer, (2) this task's
input-output examples** (half of full set), (3) currently existing strategies. Then choose
how to proceed.

--- 1. Tool buffer ---
Tool helper mode: direct_call. Helper memory is callable in strategy selection and should be
reused by direct call when applicable.

Tool buffer (callable helper functions currently available):
extract_objects, apply_recolor, apply_translate, apply_flip_horizontal,
apply_border, apply_hollow, apply_mark_center, apply_op_per_object
[helper signatures and bodies elided here -- reproduced in full in the
Tool helper definitions subsection.]

--- 2. This task's input-output examples ---
[Five 20x20 Input:/Output: example pairs elided -- same format as in the solver prompt above.]

--- 3. Currently existing strategies ---
0. Extract connected objects, choose the largest as a frame, classify other objects by whether
their bounding boxes lie strictly inside that frame, erase the frame and all outside objects,
then hollow out each inside object in place by turning its interior to color 0 while preserving
its boundary.

--- 4. Choose strategy ---
Options (include a short "reason" in your reply). You MUST pick one existing strategy -- no other
action is accepted:
B) Use an existing strategy: {"action": "select", "index": N, "reason": "brief reason"} where
N is 0 to 0

[Guidance bullets elided.]

Reply with only the JSON, no other text.
```

Response.

Selection response

```
{"action": "select", "index": 0, "reason": "Examples show keeping only the largest frame-like object,
deleting all other objects, and hollowing/removing enclosed non-frame content; this matches the
existing inside-frame filtering and hollowing strategy best."}
```

C. Strategy memory evolution over time [ARC-AGI]

C.1. How a strategy is traced across compress steps

At each compress step we run the extraction prompt (section B.4); the consolidator returns the rewritten strategy memory as a JSON list. Each list item declares its kind, one of three values:

- **retain** — keep an existing entry from the current memory; the field `from_existing` lists 1-based indices into that memory.
- **new** — distill a fresh entry from selected input tasks; the field `from_functions` lists 1-based indices into this step's input tasks.
- **merge** — emit a new entry that combines existing entries (`from_existing`) with new input tasks (`from_functions`).

Both `kind` and the two index fields come directly from the consolidator's JSON output. After parsing the response, we persist the resulting memory as a per-step JSON snapshot; each entry in the snapshot carries this step's `kind` and its associated indices.

Because both `merge` and `new` produce a fresh entry rather than editing an existing one in place, tracking how a strategy evolves over multiple steps requires walking `from_existing` pointers backwards across snapshots. Starting from a target entry, we look at its `kind`: if `kind = new`, the chain has reached its root and the walk terminates; if `kind = merge`, the index `from_existing: [j]` points to a predecessor in the previous compress step's memory; we look up the predecessor in the snapshot we persisted for that step and take it as the new target. We call a chain reconstructed by this procedure a *strategy lineage*.

C.2. Worked lineage example

We ran GPT-5.4 on a 200-task pool spanning six ARC families, with the same model serving as both solver and consolidator. We trace one strategy lineage by the procedure of section C.1, from its `new` step through its final `merge`, and show three snapshots: the chain root, an intermediate node, and the chain's final node. In the latter two, yellow shading marks spans that differ from the previous snapshot.

Chain root. The chain begins with a `new` step: the consolidator distills four inside-frame tasks into one entry.

Chain root

When to use: A large hollow rectangular frame or border encloses some smaller objects, while other objects may lie outside the frame. The output removes the frame itself and removes all outside objects, preserving only the objects strictly inside the frame. In some variants the kept inside objects are additionally mirrored left-right within their own bounding boxes.

Strategy: (1) Extract all connected non-background objects. (2) Identify the enclosing frame as the largest object, typically a hollow rectangle. (3) For every other object, compare its bounding box with the frame's bounding box; mark it as inside only if its bounding box lies strictly within the frame on all four sides. (4) Start from a copy of the input grid. (5) Erase the frame object from the grid. (6) Erase every object not strictly inside the frame, including all objects outside the frame. (7) For each surviving inside object, determine from training whether it should remain unchanged or be horizontally flipped within its own bounding box. If flipping is required, remap each occupied cell at row offset r and column offset c to row offset r and column offset $\text{width}-1-c$ inside that object's box, using the object's own color. (8) Return the full grid with only the interior objects retained.

Intermediate node. Compared with the chain root: the horizontal-flip coordinate formula at root step (7) is gone, the *When to use* clause about mirroring is gone, and a uniform-recolor step has been added in its place. The overall

scaffold — identify the frame as the largest object, classify other objects by strict bounding-box containment, erase the frame and outside objects, keep the inside objects — is preserved nearly verbatim.

Intermediate node

When to use: A grid contains one large enclosing rectangular frame and several smaller objects, some strictly inside it and others outside or touching it. The output removes the frame and discards everything not strictly enclosed, leaving only the interior objects; sometimes those kept objects are recolored to a single common color, and sometimes they are simply kept unchanged.

Strategy: (1) Extract all connected non-background objects. (2) Identify the enclosing frame, typically the largest hollow rectangular object. (3) For every other object, compare bounding boxes with the frame’s bounding box. Classify an object as inside only if its top, left, bottom, and right bounds are all strictly within the frame bounds; objects touching the frame or lying outside are not inside. (4) Create an output grid, usually starting from the input. (5) Erase the frame itself. (6) Erase every object not strictly inside the frame. (7) Keep the inside objects in their original positions. (8) **If training shows a post-processing step on the kept objects, apply it uniformly, such as recoloring all remaining cells to a single target color;** otherwise leave their internal colors unchanged. (9) Return the resulting grid.

Chain end. Compared with the previous snapshot: the recolor step is gone, replaced by a uniform-translation step at the same position, and the *When to use* now mentions *all shifted by one common offset* in line with that change. The mirror clause that was already missing in the previous snapshot is still absent.

Chain end

When to use: A grid contains one large hollow rectangular frame plus several other objects, and the output removes the frame and all objects outside it, keeping only objects strictly inside the frame. The surviving inside objects may also all be shifted by one common offset.

Strategy: (1) Extract all connected non-background objects. (2) Identify the frame as the large rectangular border object, typically the largest object. (3) For every other object, decide whether it is strictly inside by checking that its bounding box lies strictly within the frame’s bounding box on all four sides. (4) Start from a full-size output grid and erase the frame. (5) Erase every object classified as outside the frame. (6) Keep only the inside objects. (7) **If training shows a uniform translation, move every inside object by the same learned row/column offset, clearing original cells before redrawing them;** otherwise leave them in place. (8) Return the full-size grid containing only the surviving processed inside objects.

D. Examples of wrong / low-quality strategies [ARC-AGI]

Each subsection below names one failure mode and shows a verbatim entry from the strategy memory, paired with either a callout that explains highlighted spans or a held-out solver code comparison.

D.1. Vacuous abstraction

We ran GPT-5-mini on a 200-task pool spanning six families. After each task, the model could update its strategy memory or leave it unchanged; we set no cap on entry count. After task 200, the memory’s first entry read:

Strategy memory entry

Make a working copy of the input grid (list of row lists) before mutating, perform all modifications on the copy, and return the copy to avoid mutating the original input.⁽¹⁾

Why this memory is harmful

- (1) **Vacuous abstraction.** A defensive Python idiom shared by every nested-list program. Mentions no color, shape, or rule that the six families distinguish themselves by.

D.2. Concrete-vocabulary erosion

We showed GPT-5-mini one training task (“recolor the largest object”) for 50 consecutive rewrites of its memory. Round 1 still mentions “max size” — the only concrete property the task actually selects on — as a parenthetical example of the metric to compute. By round 50, the same lineage has no word left for it:

Strategy memory entry — round 1

identify and extract structured elements from input → compute a global metric (e.g., max size) → iterate elements and selectively apply targeted edits

Strategy memory entry — round 50

Find the maximum value of a derived per-object numeric attribute⁽¹⁾ and apply a uniform transformation to every object whose attribute equals that maximum.

Why this memory is harmful

- (1) **Concrete feature stripped.** Round 1’s “max size” names a property a solver can compute in cells. The round 50 phrasing names no specific feature; the entry no longer records *which* attribute to maximize.

D.3. Foreign-family injection

We presented training tasks to GPT-5.4 one at a time, each paired with its ground-truth solver code. We required the model to compress every task into a single memory entry on receipt. The compressor distilled the task into an entry whose Strategy steps mix in content from multiple families.

Strategy memory entry

When to use: A large hollow rectangular frame encloses some objects while other objects lie outside it, and the output keeps only the interior objects in their original positions while deleting the frame and all exterior objects. In the kept interior objects, a single distinguished cell is changed based on a relation to a matching object outside the frame, often when an outside object has the same shape as an inside object.⁽¹⁾

Strategy: (1) Detect the background color and extract connected non-background objects. (2) Identify the frame as the large hollow rectangle. (3) Classify every other object as inside or outside the frame by strict bounding-box containment. (4) Erase the frame and every object outside the frame. (5) For each interior object, look for an exterior object with the same shape signature, allowing translation but not reshaping; compare normalized cell sets or bounding-box-relative coordinates.^(2a) (6) If an interior object has such a matching exterior counterpart, mark the center cell of the interior object’s bounding box with the exterior object’s color.^(2b) (7) If no such counterpart exists, leave the interior object unchanged. (7) Return the full-size grid with everything else set to background.

Why this memory is harmful

- (1) **Foreign-family extension to the trigger.** The second sentence of the trigger introduces a shape-relation step that the inside-frame source task does not perform; it pre-commits the solver to look for a cross-frame shape match before any retrieval filter can reject the entry.

(2a–2b) Composite absent from every family. Spans (2a) and (2b) together form a single composite: a shape-signature lookup across the frame (2a), followed by a marker-style color write into the matched interior object’s center (2b). The lookup belongs to the group-by-shape family and the color write to the key-marker family; neither is part of the inside-frame source task, and no family in the pool prescribes this composite.

D.4. Phantom strategies from failed attempts

In a 200-task GPT-5.4 run, we exposed the strategy-update prompt to both successful and failed past attempts (each failure paired with the wrong output it produced). We expected this to let the model prune bad entries by surfacing what had not worked. The output instead distilled recurring patterns from the failed code into new entries. The strategy library at the end of the run held five entries; two were paraphrases of the same fictional rule. We reproduce the longer paraphrase below:

Strategy memory entry

When to use: A same-size output contains **glyph-like objects made of horizontal and vertical strokes**⁽¹⁾, and only one side stroke or partial row/column segment is shifted within the object’s own bounding box to **normalize the glyph**⁽¹⁾. The object is not translated as a whole and not hollowed. Often one full-width or full-height stroke stays fixed while shorter spans attached to one side move inward or to the opposite side.

Strategy: (1) Segment connected non-background objects and treat isolated marker cells separately. (2) For each candidate glyph, compute its bounding box plus row spans and column spans. (3) Identify **anchor strokes that remain fixed**, such as a full-width top or bottom bar or another span matching the final examples.⁽²⁾ (4) Detect **shorter side-attached spans that differ from the anchor-aligned placement by a constant horizontal or vertical offset**.⁽²⁾ (5) Infer from training which subset of rows or columns should move and in which direction; do not assume all non-anchor spans move. Often the moved spans are the interior partial strokes while an outer full bar stays unchanged. (6) **Clear the original glyph and redraw it inside the same bounding box using the unchanged anchor spans plus the shifted partial spans**.⁽³⁾ (7) Apply the same **local stroke-shift rule**⁽¹⁾ to each target glyph class, preserving color and overall bounding-box location, and leave unrelated objects unchanged.

Why this memory is harmful

- (1) Fictional family and operation traced to failure code.** Neither “glyph-like” (the family) nor “normalize the glyph” / “stroke-shift rule” (the operation) matches anything in the six families the run contained; the consolidator produced both. The lineage traces to a recurring fragment in failed solver code: `out[r][min(cols)] = 0` (“erase the leftmost cell of this row”). The strategy-update step read this fragment as evidence of an intentional row-level edit rule; the font-rendering metaphor (a glyph being normalized by a stroke shift) is the abstraction it produced.
- (2) Structural vocabulary borrowed from neighboring families.** “Anchor stroke” and “side-attached span” resemble the geometric vocabulary used by the inside-frame and compose-horizontal families (frame edges, panel borders), but the entry applies these terms to a different referent — internal parts of a single object — where no real family does so.
- (3) Operational scaffolding for a non-existent rule.** Step (6) gives the solver concrete, copy-paste-ready instructions (“clear the original glyph”, “redraw inside the same bounding box”), but the instructions are wrong.

D.5. Single-strategy collapse on multi-family compression

The cases above describe entries whose content is itself problematic. This subsection describes a different failure with two parts: (i) compression collapses multi-family training tasks into one single-family playbook; (ii) retrieval applies that entry verbatim to held-out tasks from other families. The entry text itself is internally consistent and appropriate

for its source family.

Example 1: 19 tasks revisited 10 times collapse to a single inside-frame plus hollow strategy. We presented a fixed pool of 19 tasks, spanning the six families, to GPT-5.4 10 times. The model viewed one task at a time alongside its ground-truth solver code, and updated its memory after each task. After 10 passes (190 steps), the memory contained a single entry:

Strategy memory entry

Extract connected objects, choose the largest as a frame, classify other objects by whether their bounding boxes lie strictly inside that frame, erase the frame and all outside objects, then hollow out each inside object in place by turning its interior to color 0 while preserving its boundary.

On a held-out task whose true rule was “erase every object of one specific color” — unrelated to the inside-frame pattern in the entry — the same model produced:

Held-out solver code with memory (color-removal task)

```
def solve(grid):
    # Keep only non-frame objects; frames are the largest
    # connected components (color 6 in the examples).
    # Then hollow the kept objects.
    h = len(grid); w = len(grid[0]) if h else 0
    objs = extract_objects(grid)
    if not objs:
        return [row[:] for row in grid]

    frame = max(objs, key=lambda o: o["size"])
    fb = frame["bbox"]

    out = [[0] * w for _ in range(h)]
    for obj in objs:
        if obj is frame: continue
        bb = obj["bbox"]
        inside = (bb["top"] > fb["top"]
                  and bb["bottom"] < fb["bottom"]
                  and bb["left"] > fb["left"]
                  and bb["right"] < fb["right"])
        if inside:
            for r, c in obj["cells"]:
                out[r][c] = obj["color"]

    for obj in extract_objects(out):
        apply_hollow(out, obj, fill_color=0)
    return out
```

The first comment reproduces the entry’s framing verbatim. The second comment records the active color from the examples. The code below treats color 6 as the frame color rather than as the color to remove. The code follows the entry’s frame-and-inside structure: it picks a frame by size, copies only strictly-interior objects onto a blank canvas, and hollows them in place. On this input no color-3 marker passes the strict-inside check, so the canvas stays empty. The same model without strategy memory solved the task in eight lines:

Same model on the same task, no strategy memory

```
def solve(grid):
    # The transformation removes every connected component of color 6
    # and leaves all other cells unchanged.
    out = [row[:] for row in grid]
    for obj in extract_objects(grid):
        if obj["color"] == 6:
```

```
for r, c in obj["cells"]:
    out[r][c] = 0
return out
```

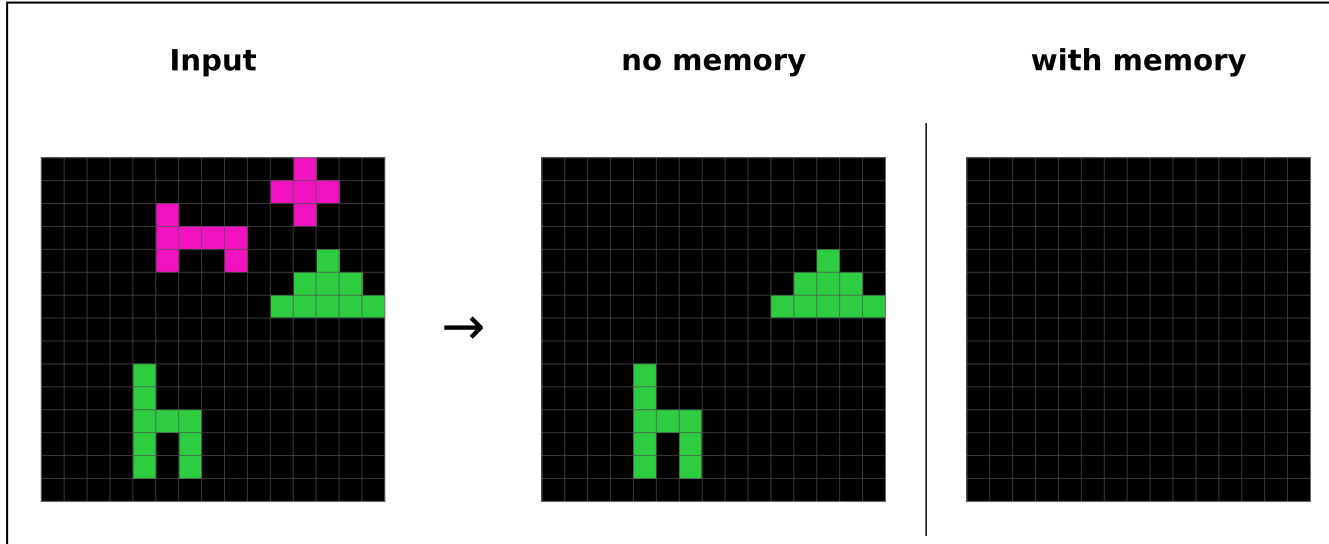


Figure 14: Example 1, held-out task. The input contains a hollow color-6 frame and several smaller color-3 markers. The same GPT-5.4 solver passes this task without strategy memory (center, 10/10) by erasing every color-6 component and keeping the markers; with memory (right, 0/10) the solver instead executes the entry’s frame-and-hollow playbook and produces an empty grid.

Example 2: 250 tasks streamed once collapse to a group-by-shape strategy. In a separate run, we presented 250 training tasks to GPT-5.4 one at a time, each paired with its ground-truth solver code. After each task the model chose whether to update the memory. Training tasks again spanned the six families. After 250 steps the memory contained two entries, both from the group-by-shape family. Each entry prescribed partitioning objects by translation-normalized shape signature, keeping the most frequent signature class, and applying a decoration to the survivors. The other five families (largest-object, color-property, key-marker, inside-frame, compose-horizontal) had no entry in the memory.

Strategy memory entry

When to use: The grid contains several disconnected non-background objects. The output keeps only the objects whose translation-normalized shape occurs most often, erases the rest, and applies a single decoration to each survivor.

Strategy: (1) Extract all connected non-background objects. (2) For each object, compute a position-independent shape signature by translating its cells so its topmost-leftmost cell becomes the origin, then sorting the relative coordinates. (3) Count how many objects share each signature; pick the majority signature, breaking ties by first appearance. (4) Starting from a copy of the input, erase every object whose signature is not the majority signature. (5) Keep all objects with the majority signature unchanged in place and color. (6) Apply the task’s required common decoration to each survivor; in one common variant, draw a one-cell outline around the object using a marker color without overwriting the object itself.

On a held-out task from the largest-object family — true rule “keep only the largest connected component, erase the rest” — the same model with this memory produced:

Held-out solver code with memory (keep-largest task)

```
def solve(grid):
    objs = extract_objects(grid)
    if not objs:
        return [row[:] for row in grid]

    def signature(obj):
        cells = obj["cells"]
        top = min(r for r, c in cells)
        left = min(c for r, c in cells)
        return tuple(sorted((r - top, c - left) for r, c in cells))

    # Count normalized shapes, regardless of color/position.
    counts = {}
    first_seen = {}
    for i, obj in enumerate(objs):
        sig = signature(obj)
        counts[sig] = counts.get(sig, 0) + 1
        if sig not in first_seen:
            first_seen[sig] = i

    # Keep the most frequent shape; break ties by first appearance.
    best_sig = min(
        counts,
        key=lambda s: (-counts[s], first_seen[s])
    )

    h, w = len(grid), len(grid[0])
    out = [[0] * w for _ in range(h)]

    for obj in objs:
        if signature(obj) == best_sig:
            color = obj["color"]
            for r, c in obj["cells"]:
                out[r][c] = color

    return out
```

The solver transcribes the entry's selection logic into Python: `signature()` implements step (2), the dictionary count implements step (3), and `best_sig` implements step (4). The downstream operation — writing each kept object's cells onto a blank canvas — is identical to what the same model produces without memory, shown next:

Same model on the same task, no strategy memory

```
def solve(grid):
    objs = extract_objects(grid)
    if not objs:
        return [row[:] for row in grid]
    # Keep only the largest object(s); in all examples the output is exactly
    # the maximal connected component and everything else is removed.
    max_size = max(obj["size"] for obj in objs)
    out = [[0 for _ in row] for row in grid]
    for obj in objs:
        if obj["size"] == max_size:
            color = obj["color"]
            for r, c in obj["cells"]:
                out[r][c] = color
    return out
```

Without memory the model identifies the correct selection criterion (`obj["size"] == max_size`). With memory the selection criterion is replaced by the entry's `signature(obj) == best_sig`. The action that follows the selection is unchanged. Replacing the selection criterion alone, while keeping the action identical, drops the solver from 10/10 to 2/10 on this task's 10 test grids.

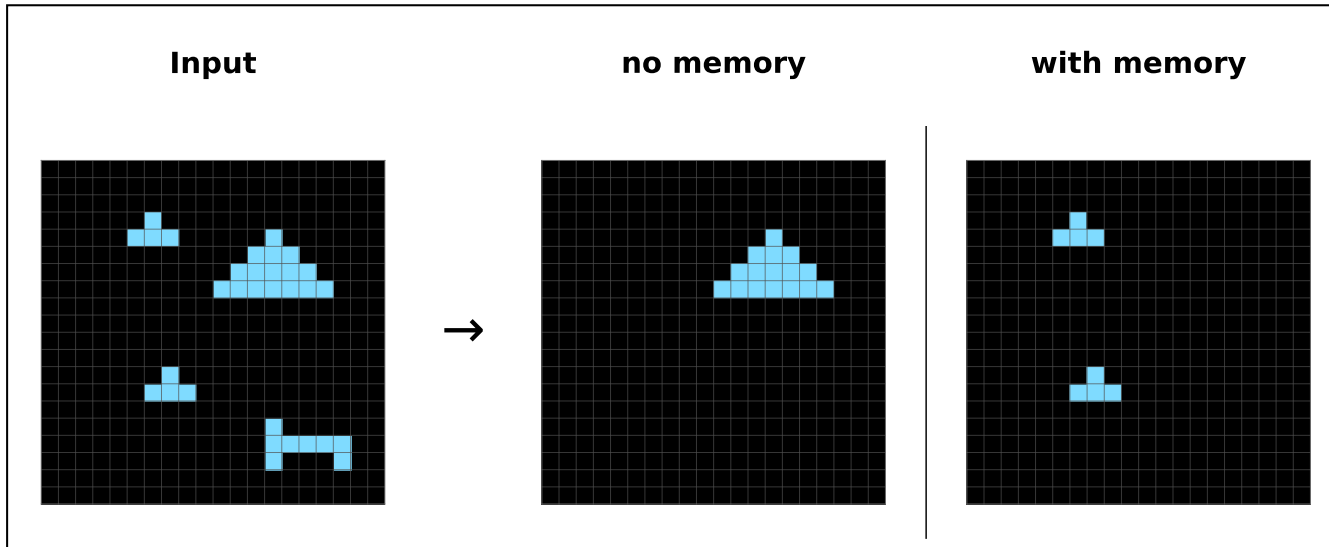


Figure 15: Example 2, held-out task. The input contains one large unique triangle (top-right), one medium unique H-shape (bottom-right), and two small same-shape T-shapes (top-left and mid-left). The same GPT-5.4 solver passes this task without memory (center, 10/10) by keeping only the largest object; with memory (right, 2/10) the solver keeps only the two same-shape T-shapes, since the entry’s selection criterion is shape-signature majority rather than size.

E. Examples of useful / well-formed strategies [ARC-AGI]

Below are four memory entries that share one property: their procedural body is concrete and directly operational — each step names an action the solver can translate into a Python helper, and the directions, boundaries, and object selections it depends on are stated specifically rather than left implicit. Yellow highlights mark the phrases that most directly drive this property.

Strategy memory entry — compose-horizontal (operation left as a slot)

When to use: The task has two separate input grids of the same height, and the output width is exactly the sum of their widths, indicating horizontal concatenation. The left half of the output is just the first input, while the right half is the second input after a uniform transformation applied only there: either every non-background cell is recolored to one fixed color, or each connected component keeps its shape but receives a center marker at its bounding-box midpoint.

Strategy: (1) Verify that the output is formed by placing the first input on the left and the second input on the right with no vertical offset. (2) Build the output by horizontally concatenating corresponding rows of the two inputs. (3) Determine what extra rule applies only to the right half by comparing examples. (4) If the right-half rule is uniform recoloring, replace every non-background cell from the second input by the designated output color while preserving all positions and shapes. This can be done cellwise or per connected component. (5) If the right-half rule is center marking, extract each non-background 4-connected component in the second input, compute its bounding box, and find the bbox center using the midpoint row and midpoint column. Shift that center into the concatenated output’s right half and write the marker color there, leaving the rest of the component unchanged. (6) Keep the left half identical to the first input throughout.

Strategy memory entry — largest-object

When to use: The grid contains several disconnected objects on the same background, and the output changes only the object or objects with the greatest area/cell count while leaving all smaller objects untouched. The transformation on the selected objects may be marking a special point, recoloring the whole object, or mirroring the object within its own bounding box.

Strategy: (1) Extract all connected non-background objects. (2) For each object, compute its size as the number of occupied cells. (3) Find the maximum size across all objects; if several objects tie for largest, select all of them. (4) Copy the input grid to the output. (5) Apply the task-specific operation only to the selected largest objects, preserving their positions: for example, recolor every cell of each selected object, mark the object’s center cell if the examples show a single inserted marker, or horizontally mirror the object’s occupied cells within its own bounding box. (6) Leave every non-selected object and all background cells unchanged.

Strategy memory entry — inside-frame

When to use: A grid contains one large hollow rectangular frame and several separate objects both inside and outside it. The output keeps only the objects strictly enclosed by the frame, removes the frame itself, removes every outside object, and leaves the kept inner objects unchanged in their original positions on a same-size grid.

Strategy: (1) Find all connected non-background objects. (2) Identify the frame as the large hollow rectangle, typically the largest object by size and with an empty interior. (3) For every other object, compare its bounding box to the frame’s bounding box; keep only objects whose top, left, bottom, and right bounds lie strictly inside the frame bounds, not touching the frame border. (4) Start from a copy of the input or from an empty same-size grid. (5) Erase the frame cells. (6) Erase every object not strictly inside the frame. (7) Preserve each inside object exactly as is: same shape, same color layout, same absolute position. (8) Leave all remaining cells as background.

Strategy memory entry — key-marker (earlier extraction schema)

Use the top-left cell as a preserved control marker; for every other object, choose between two transforms according to whether that marker matches a designated color.

F. Additional Experiment Details

F.1. ALFWorld

Training data

We use trajectories on the six standard ALFWorld task types, generated by gpt-4.1 or gpt-5-nano (see table 7):

- pick_and_place
- pick_clean_then_place_in_recep
- pick_two_obj_and_place
- look_at_obj_in_light
- pick_cool_then_place_in_recep
- pick_heat_then_place_in_recep

We use script-normalized task names; the upstream ALFWorld task type for pick_and_place is pick_and_place_simple, with the _simple suffix stripped during data preparation.

“ $n=64$ ” and “ $n=128$ ” below refer to the number of trajectories *per task type*, so a $n=64$ set contains $64 \times 6 = 384$ trajectories in total. All rollouts use max_steps=50.

Three training-data variants are used across the experiments (table 7):

| Variant | generator | per type | total | filtering |
|----------------------------|------------|----------|-------|------------------------------|
| <i>n</i> =64 golden | gpt-4.1 | 64 | 384 | only success-trajectory kept |
| <i>n</i> =128 golden | gpt-4.1 | 128 | 768 | only success-trajectory kept |
| <i>n</i> =64 with_failures | gpt-5-nano | 64 | 384 | none (mixed success/failure) |

Table 7: The three ALFWorld training-data variants differ on two axes: total size (384 vs 768 trajectories) and reward filtering. The two golden variants retain only success trajectories from gpt-4.1 rollouts; *with_failures* retains both successes and failures from gpt-5-nano rollouts and is used for probes that test consolidation behaviour under imperfect inputs.

After collection, every raw trajectory is converted into a structured memory item via the abstraction step of the SkillRL pipeline. We use gpt-5.4 as the abstraction LLM. The abstraction makes 3 LLM calls per trajectory and produces one item with three fields: `contextual_description`, `refined_trajectory`, and `strategic_guidelines`.

Two-task-type probe set

For one probe experiment we use a focused training subset that pairs two task types: `pick_and_place` (focal) and `pick_clean_then_place_in_recep` (probe). The base memory at every probe point is a single-type consolidation that fixes the per-stage task type to `pick_and_place` and samples 32 abstracts per stage from the *n*=64 abstract pool; snapshots are taken at consolidation stage $S \in \{1, 20, 50, 100\}$. From each snapshot we run one additional consolidation iteration with 32 abstracts. The management model is gpt-5-nano and it has a max capacity of memory, which is 50.

Evaluation set

We evaluate on `eval_id_balanced_48`, a 48-episode subset of ALFWorld’s in-distribution test split with 8 episodes per task type for the 6 task types listed above.

Metric

Each ALFWorld episode has a binary success indicator (`env reward = 1` when the agent satisfies the goal within `max_steps=50`, else 0). We report **wins/48** (the count of successful episodes) and the corresponding success rate in $[0, 1]$ as the per-condition score on this evaluation set.

F.2. WebShop

Training data

WebShop training data come in two variants (table 8).

Golden 128. We sample 128 trajectories (`seed = 42`) from the WebShop subset of the AgentGym AgentTraj-L corpus, filtered to `score = 1.0` (perfect-reward demonstrations only). We replay these trajectories through our WebShop environment instance (`num_products=1000`) using a replay script that re-executes each AgentGym action sequence to obtain matching raw trajectories. The 128 sampled tasks are drawn sparsely from AgentGym’s success-filtered train pool; their underlying WebShop session identifiers span the range 72–6772 (sparse; 128 distinct IDs). This is the training set used for the AWM-style Workflow 8 ablation in section 1.1.

Non-golden 128. We use a separate, contiguous block of 128 WebShop sessions (session IDs 1500–1627), disjoint from the test pool $[0, 500)$. On these sessions we collect 128 trajectories generated by gpt-5.4-mini directly, with no reward filtering. Most resulting trajectories have `score < 1.0` (table 8). This set tests how memory consolidation behaves when the input training trajectories are themselves imperfect.

| Variant | source / generator | session IDs | filtering / note |
|-------------------|---------------------------|-----------------------------|--|
| golden 128 | replay from AgentGym’s | 72–6772
(sparse) | only score = 1.0 kept;
all expert demonstra-
tions |
| non-golden
128 | gpt-5.4-mini self-rollout | 1500–1627 (con-
tinuous) | none; 20/128 with
score = 1.0 |

Table 8: Composition of the WebShop training-data variants. The two variants use *different* sets of 128 WebShop session IDs and serve different probes: golden 128 supplies expert demonstrations for AWM-style consolidation; non-golden 128 supplies a mixed-success training set for studying consolidation under imperfect inputs.

After collection, raw trajectories are abstracted into structured memory items via the same SkillRL abstraction pipeline used for ALFWorld.

Evaluation set

We evaluate on `test50_seed42_no_leak.json`, a 50-session subset of WebShop’s test pool (`num_products=1000`, `seed = 42`) drawn from the index range $[0, 500)$. The 13 training-set session IDs (all from **Golden 128**) that fall within $[0, 500)$ are excluded from the sampling pool before drawing. Final session indices span 3–467.

Metric

Each WebShop episode terminates with a reward in $[0, 1]$ computed from attribute match between the purchased product and the instruction (a perfect match yields reward 1.0). If the agent never issues a `click[Buy Now]` action and the episode ends without a purchase (e.g., the agent times out at `eval_max_steps`), the reward is 0. We report two quantities per condition: **wins/50** (the count of episodes with reward = 1.0) and **mean reward** (arithmetic mean of episode rewards over the 50 evaluation sessions).

F.3. ScienceWorld

We select 15 medium-difficulty tasks from ScienceWorld for the memory consolidation experiments, where each selected task satisfies an average optimal action sequence length between 20 and 100. In total, these tasks contain **485** rollout cases. The task names and the number of rollout cases for each task are shown in Table 9.

Our ScienceWorld experiments are based on the CLIN setting, with several adaptations for studying long-horizon memory consolidation. Following CLIN, each rollout is presented together with an *evaluation report*⁽¹⁾, which provides feedback about the success or failure of the corresponding attempt. We also follow *the CLIN-style constrained memory format*⁽²⁾, where each memory is written as a unified natural-language rule using one of a small set of relation phrases, such as `MAY BE NECESSARY to`, `SHOULD BE NECESSARY to`, `MAY BE CONTRIBUTE to`, and `DOES NOT CONTRIBUTE to`.

Compared with the original CLIN update procedure, we introduce several modifications. First, we maintain a bounded memory bank and explicitly inform the LLM of the maximum memory-bank size and FIFO replacement policy, as highlighted in *the memory-capacity instruction*⁽³⁾. Second, instead of only adding new memories, we allow multiple memory-bank operations, including *delete*⁽⁴⁾, *replace*⁽⁵⁾, *add*⁽⁶⁾, and *retain*⁽⁷⁾. Third, we feed rollouts to the LLM in mini-batches and ask the model to compare the buffered rollouts jointly before editing the memory bank, as shown in *the mini-batch consolidation instruction*⁽⁸⁾. Finally, after each full pass over all rollouts for a given number of consolidation rounds, we evaluate the current memory bank on a fixed ScienceWorld test set. This test set contains **44** cases sampled from the same 15 selected tasks, with 2–4 cases per task depending on the number of available variants.

Memory update prompt template for ScienceWorld

You are given BUFFERED ROLLOUTS, which are multiple recent rollouts that have NOT been summarized yet. Each rollout is presented as a surface-level summary (compressed narrative), and is accompanied by an EVALUATION REPORT⁽⁷⁾ indicating the success of that attempt.

You are also provided with PREVIOUS LEARNINGS collected from the agent’s earlier attempts across interleaved tasks and environment variants. Some memories may be specific to one task or one environment configuration, while others may generalize across related tasks or across multiple environments. TASK indicates the task description. EPISODE indicates when that learning was created. PREVIOUS LEARNINGS also have EVALUATION REPORTS⁽¹⁾ indicating how successful the respective attempt was.

Your job is to UPDATE the FULL MEMORY BANK now, based on the BUFFER of recent rollouts that have NOT been summarized yet. You may: (a) delete outdated memories⁽⁴⁾, (b) replace several memories with a NEW set of memories⁽⁵⁾, (c) add new memories based on the buffered rollouts⁽⁶⁾, (d) retain selected buffered rollouts as concrete case records in memory⁽⁷⁾.

In this memory-update call, process this MINI-BATCH of <buffered_rollout_count> buffered rollouts together (configured batch size k=<pool_batch_size>). Compare the rollout summaries jointly before editing the MEMORY BANK. If several rollouts support the same general lesson, prefer a single compact memory that captures the shared pattern rather than duplicating near-identical memories. If the rollouts disagree, preserve only the parts that are clearly supported by the evidence.⁽⁸⁾

The MEMORY BANK can hold at most <memory_bank_max_size> entries and currently contains <current_size> entries. The MEMORY BANK is maintained with a FIFO policy: if it grows beyond capacity, the oldest entries will be dropped first.⁽³⁾

When adding or replacing memories, ONLY use rollouts you explicitly select from the buffer (used_rollout_ids). Do NOT use or rely on any rollout outside the buffer.

Because the training schedule alternates across tasks and variants, prefer memories that capture reusable preconditions, transferable mistakes, or environment knowledge that can generalize to related problems. Keep a memory narrowly task-specific only when the evidence strongly suggests it.

When writing new memories, use the SAME format as the examples below: Each numbered item in the summary can ONLY be of the form: X MAY BE NECESSARY to Y. X SHOULD BE NECESSARY to Y. X MAY BE CONTRIBUTE to Y. X DOES NOT CONTRIBUTE to Y.⁽²⁾

Use retain when a rollout’s raw narrative is itself worth keeping as a concrete case record. Use add / replace when you want abstracted lessons instead. A single rollout may support both add and retain in the same response.

Return ONLY a JSON object with this schema:

```
{ "actions": [ {"type": "delete", "ids": [int, ...]}, {"type": "replace", "ids": [int, ...],  
  "new_memories": ["<memory1>", "<memory2>", ...], "used_rollout_ids": [int, ...]}, {"type":  
  "add", "new_memories": ["<memory1>", "<memory2>", ...], "used_rollout_ids": [int, ...]}, {"type":  
  "retain", "rollout_ids": [int, ...]} ] }
```

Notes: - IDs refer to the MEMORY BANK entry IDs shown below. - used_rollout_ids refer to BUFFERED ROLLOUT IDs shown below, and indicate which rollouts were used to create the new memories. - Include an empty actions list only if you judge that no edit is needed this round. - For each retained rollout, the corresponding SURFACE SUMMARY will be inserted into the MEMORY BANK verbatim as its own memory entry.

F.4. AppWorld

Training data

We use 140 trajectories collected on AppWorld’s official train and dev splits (Trivedi et al., 2024) as the shared training set across all memory-induction methods evaluated in this work.

The 90-task train segment uses AppWorld’s officially released ReAct baseline trajectories, produced with GPT-4o (gpt-4o-2024-05-13). The 50-task dev segment uses ReAct trajectories collected with Qwen3.5-27B on a stratified-by-difficulty sample of the dev split, run under AppWorld’s standard ReAct interface. Composition is summarized in table 10. The mean Task Goal Completion (TGC) over the combined 140 trajectories is 61.4% (86/140): 52.2% on the train segment and 78.0% on the dev segment.

Table 9: ScienceWorld tasks used for memory consolidation.

| Task name | # Rollout cases |
|--|-----------------|
| boil | 28 |
| change-the-state-of-matter-of | 28 |
| chemistry-mix | 30 |
| chemistry-mix-paint-tertiary-color | 34 |
| freeze | 28 |
| grow-fruit | 40 |
| grow-plant | 40 |
| identify-life-stages-1 | 12 |
| measure-melting-point-known-substance | 40 |
| measure-melting-point-unknown-substance | 40 |
| melt | 27 |
| power-component-renewable-vs-nonrenewable-energy | 18 |
| test-conductivity | 40 |
| test-conductivity-of-unknown-substances | 40 |
| use-thermometer | 40 |
| Total | 485 |

For methods that require gold-quality input, we additionally collect a parallel set of 140 trajectories on the same task IDs by replaying AppWorld’s released ground-truth solutions through the environment API, yielding a 100%-pass training set with identical task IDs and ordering.

| Subset | N | Source split | Trajectory backend |
|---------------|-----|---------------------------|--------------------|
| Train segment | 90 | AppWorld train (90 tasks) | GPT-4o |
| Dev segment | 50 | AppWorld dev (57 tasks) | Qwen3.5-27B |
| Combined | 140 | — | — |

Table 10: Composition of the 140-trajectory AppWorld training set.

Each training datapoint consists of a task identifier, a final pass/fail outcome, and a Qwen3.5-27B-generated prose summary of the full ReAct trajectory (thoughts, actions, and observations of all steps), used in place of the raw token-level trajectory to keep per-entry length tractable for prompt-time concatenation.

Evaluation set

We evaluate on a 100-task subset drawn from AppWorld’s two official test splits (`test_normal` with 168 tasks and `test_challenge` with 417) by proportional stratified sampling on difficulty: 50 tasks from each split, `seed = 42`, with no overlap. All conditions are evaluated on the same 100 task IDs.

Metric

Each AppWorld task is paired with a set of Python unit tests. The official evaluator replays the agent’s trajectory against the resulting database state and runs these tests; a task is counted as a success only when all of its unit tests pass. Following AppWorld’s official protocol, we report Task Goal Completion (TGC) as the sole metric: the percentage of evaluation tasks whose success indicator equals one. All TGC values reported in the main tables are arithmetic means over the 100 evaluation tasks.

G. Case studies of memory failure modes

G.1. Erosion: good memory degrading over time

Setup. The memory under study is produced by a 200-stage iterative consolidation run on ALFWorld. The base memory at stage 0 is a 10-item seed obtained by passing all 768 abstracts of the $n=128$ golden ALFWorld training set (section F.1) through a single gpt-5.4 consolidation call that emits a free-count list of task-type-agnostic insights; the model picks the count, with no fixed target. Each subsequent stage feeds gpt-5-nano a fresh mini-batch of 6 abstracts (1 per task type, drawn from the $n=64$ subset of the same six-type pool) together with the current memory; the management model rewrites the entire memory at every stage and forces overwrites rather than appends. Besides, the memory capacity is 50. We evaluate the consolidated memory at every 20 stages on the held-out balanced ALFWorld split (defined in section F.1; $n=48$ episodes) using three Qwen3.5 sizes as rollout policies — Qwen3.5-4B, Qwen3.5-9B, and Qwen3.5-27B — with each model’s reasoning mode disabled (the “no-thinking” setting). The full memory is supplied in the prompt context at every step, so the rollout sees every item in every snapshot.

The erosion curve. fig. 16 plots wins/48 for the three rollout sizes at all evaluated stages, against the corresponding no-memory baselines (15/48 for both 4B and 9B; 19/48 for 27B). All three sizes show the same qualitative pattern: an early peak followed by a general decline (with small recoveries) to stage 200. The 4B and 9B rollouts each gain a small bump at stage 20 (35/48 wins), then erode to 15/48 (4B) and 16/48 (9B). The 27B rollout’s peak is at stage 0 (41/48), so for the 27B rollout consolidation never helps; it erodes to 22/48 by stage 200. By stage 200 the gap to the no-memory baseline collapses for the 4B and 9B rollouts ($\Delta=0$ and $\Delta=1$ respectively); the 27B rollout retains a $\Delta=3$ gap above its baseline (22/48 vs. 19/48). Memory size stays bounded at 38–50 items by the cap throughout, so the erosion is in content quality, not item count.

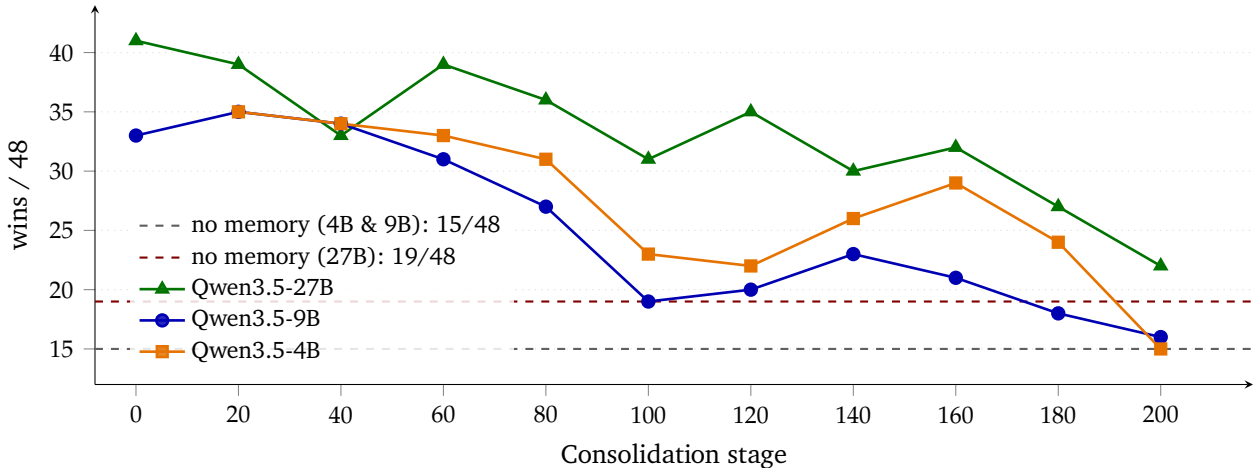


Figure 16: Iterative consolidation erodes the memory’s value across three rollout sizes. By stage 200, the 4B and 9B rollouts collapse to within 1 win of their no-memory baselines (15 vs. 15; 16 vs. 15); the 27B rollout retains a 3-win gap above its baseline (22 vs. 19). The 4B and 9B rollouts peak early (stage 20, 35/48 wins each) before declining; the 27B rollout’s peak is the un-consolidated base seed at stage 0 (41/48), so consolidation never helps it. Memory size is bounded by `max_memory_size=50` throughout, so the loss is in content quality, not item count.

What is in the memory at the peak. At stage 20 the 42 items each cover a distinct decision-relevant insight. We reproduce items 0–5 verbatim below; items 6–41 expand on this cover set with concrete recipes for heat/cool/clean tasks, look-at-light tasks, and multi-object planning.

Stage 20 memory (peak, 42 items) — items 0–5 verbatim

Item 0. Most ALFWorld tasks reduce to a small set of reusable plans. For plain pick-and-place: find object, pick it up, go to destination, place it. For cool/heat/clean tasks: find object, pick it up, apply the required state change at the correct appliance/fixture, then move it to the goal receptacle. For look-at-obj-in-light tasks: get the target under or near the desk lamp, turn on/use the desk lamp, then examine or hold the target there. Recognizing the task type early lets you prioritize the right fixtures: fridge for cooling, microwave for heating, sinkbasin for cleaning, desk lamp for light-examination.

Item 1. When the object location is unknown, systematic search beats wandering. Common sources recur by room type: kitchen objects often appear on countertop, dining table, in fridge, cabinet, drawer, sinkbasin, stoveburner, or microwave; living-room objects often appear on sofa, armchair, coffee table, side table, desk, shelf, dresser, or TV stand; bathroom objects often appear on toilet, toilet paper hanger, hand towel holder, bathtub basin, countertop, cabinet, or drawer. Open closed containers when needed, and once you find one instance of the target type, revisit that source because the second instance is often in the same place.

Item 2. If the goal location is a closed container like a drawer, cabinet, fridge, microwave, or safe, opening it early can simplify the rest of the task. This avoids carrying the object to the destination only to discover it is closed, and it is especially helpful in two-object tasks because the container remains ready for repeated placements. The same applies when the destination is also the object source: open once, retrieve object, do the state change, then return and place.

Item 3. Cooling and cleaning are usually simpler than heating: once you hold the target object, go to the fridge or sinkbasin and apply ‘cool X with fridge’ or ‘clean X with sinkbasin’. You often do not need to insert the object into the appliance/container first. This means a strong default is: pick up target, go to the relevant fixture, apply the state change directly, then place at destination.

Item 4. Many state-change tasks start with the target object already on or in the goal receptacle. In these cases, the object still must be picked up, changed to the required state, and then placed back. Examples include cooling an object already on a dining table or in a coffee machine, heating an object that ends up back in the fridge or dining table, and cleaning an item that returns to the same surface. Do not stop just because the object is already at the destination.

Item 5. The robust default for ‘pick two obj and place’ is: find one target, pick it up, carry it to the destination, place it, return for the second, and repeat. If both objects are visible and inventory permits, carrying both can help, but many successful trajectories simply shuttle one at a time. Once a source location is known to contain multiple copies, revisit it directly instead of re-searching the whole room.

What is in the memory after erosion. At stage 200 the 38 items collapse onto a small number of overlapping templates. By keyword scan, 22 of the 38 items mention the desk lamp, 28 mention multi-object handling, and 33 mention systematic room-by-room search; the templates overlap heavily (most items mention more than one). Items 3 and 5 are byte-identical. The concrete meta-strategies of stage 20 — the cooling-vs-heating asymmetry of item 3, the shuttle default of item 5, the recognize-task-type-early principle of item 0 — do not appear in concrete actionable form in any of the five representative items below, and a full scan of the 38 items finds none that retain the cooling-vs-heating asymmetry as a separable insight.

Stage 200 memory (eroded, 38 items) — representative items

Item 3. Pattern: For a single target inside a container, open the container to access the object, pick it into the inventory, move directly to the destination surface, and place the object there. ⁽¹⁾ Planning: always consider planning/visibility by keeping the destination in view or planning a route that preserves visibility; open destinations early to simplify access. Unknown-location handling: if location is unknown, perform a systematic room-by-room search to locate and acquire the target before planning. Example: retrieve a candle from a cabinet and place it in the toilet.

Item 5. Pattern: For a single target inside a container, open the container to access the object, pick it into the inventory, move directly to the destination surface, and place the object there. ⁽¹⁾ Planning: always consider planning/visibility by keeping the destination in view or planning a route that preserves visibility; open destinations early to simplify access. Unknown-location handling: if location is unknown, perform a systematic room-by-room search to locate and acquire the target before planning. Example: retrieve a candle from a cabinet and place it in the toilet.

Item 8. Always illuminate targets with the desk lamp during planning and fetching ⁽²⁾; relocate targets onto a lamp-hosting surface or move the lamp to the target surface and switch it on; use lamp-assisted look-at to maximize visibility and minimize wandering; Planning/visibility: keep targets in view during planning and fetching; open destinations early; In multi-object tasks, plan the fetch order to minimize wandering and travel distance; plan their final arrangement on the destination surface when carrying two objects; Unknown-location handling: perform a systematic room-by-room search to locate targets and the lamp, acquire targets, and carry them to the destination while preserving visibility of destinations during planning.

Item 9. Always illuminate targets with the desk lamp during planning and fetching ⁽²⁾; relocate targets onto a lamp-hosting surface or move the lamp to the target surface and switch it on; use lamp-assisted look-at to maximize visibility and minimize wandering. Planning/visibility: keep targets in view during planning and fetching; open destinations early. Unknown-location handling: perform a systematic room-by-room search to locate targets and the lamp, acquire targets, and carry them to the destination while preserving visibility of destinations during planning. For two-object tasks: plan final arrangement on the destination surface when carrying two objects; if the inventory can carry two objects, carry two; otherwise place the first object, fetch the second, and then place it. Distractors: set aside held distractors to free space.

Item 10. Pattern: Always illuminate targets with the desk lamp during planning and fetching ⁽²⁾; relocate targets onto a lamp-hosting surface or move the lamp to the target surface and switch it on; use lamp-assisted look-at to maximize visibility and minimize wandering. Implementation: 1) Before approaching a target, ensure the desk lamp is on. 2) If needed, reposition the lamp or move the target onto a surface that can host the lamp to achieve optimal lighting. 3) Use lamp-assisted look-at to orient yourself and inspect the target under the light, reducing uncertainty and stray movement. 4) Planning/visibility: keep the target in view during planning and fetching; open destinations early to simplify access. Unknown-location reinforcement: if location is unknown, perform a systematic search to locate the target and the lamp, acquire targets, and carry them to the destination while preserving destination visibility during planning.

Erosion modes

- (1) **Byte-identical duplicate.** Items 3 and 5 of stage 200 (section G.1) are character-for-character identical, occupying two of the 50 cap slots with the same content. The pipeline applies no post-hoc deduplication step, so gpt-5-nano emitting the same string twice within one consolidation stage produces two stored copies.
- (2) **Paraphrase cluster (desk lamp directive).** Items 8, 9, and 10 of stage 200 each open with a near-verbatim restatement of “always illuminate targets with the desk lamp during planning and fetching”, followed by progressively rephrased downstream advice. 21 of the 38 items at stage 200 contain this opener. The desk lamp directive only applies to look-at-in-light tasks, which are 8/48 of the eval split; the memory devotes 21/38 (>55%) of its prompt-context slots to it, crowding out the diverse meta-strategies of stage 20.
- (3) **Lost meta-strategy.** The cooling-vs-heating asymmetry of stage 20 item 3 (Cooling and cleaning are usually simpler than heating — the agent can apply cool X with fridge or clean X with sinkbasin without inserting the object first) is absent from every item at stage 200. The 50-item cap evicted it to make room for paraphrases of the desk lamp template. The shuttle default for two-object tasks of stage 20 item 5 (The robust default for ‘pick two obj and place’) is also absent in this concrete actionable form, replaced by 15 paraphrases of a high-level “carry two in a single trip” directive that does not specify the fallback.

G.2. Collapsed memory: single-stage merge to one item

Setup. The iterative-consolidation pipeline matches section G.1: gpt-5-nano management model, memory capacity is 50 items, the same six ALFWorld task types, and the same gpt-5.4 abstraction step. The per-stage data sampling differs in one respect. Each stage feeds the manager a mini-batch of 6 trajectories drawn from a single (randomly chosen) task type, rather than a flat sample across all six types. Rollout policies are the three Qwen3.5 sizes (Qwen3.5-4B, Qwen3.5-9B, Qwen3.5-27B); the evaluation split is the same balanced ALFWorld split as in section G.1. We focus on the single-stage transition $S=168 \rightarrow S=169$ in this run.

The collapse event. At stage 168 the memory contains 50 items totalling 48,506 characters (i.e., the cap is saturated, average ~ 970 characters per item). One consolidation step later, at stage 169, the memory contains a *single item* of 1,960 characters. The manager has merged 50 structured items into one numbered “unified loop” that tries to cover every ALFWorld task type at once. The collapse is concentrated in this single stage. The preceding stage 167 has 49 items; the following stage 170 already begins to rebuild (2 items, 3,410 chars). table 11 reports the rollout-level effect of this single-stage transition under the three Qwen3.5 sizes, against the corresponding no-memory baselines. section G.2 reproduces the single stage-169 item verbatim.

| Rollout policy | no memory | stage 168 (50 items) | stage 169 (1 item) | Δ |
|----------------|-----------|----------------------|--------------------|----------|
| Qwen3.5-4B | 15/48 | 35/48 | 29/48 | −6 |
| Qwen3.5-9B | 15/48 | 36/48 | 26/48 | −10 |
| Qwen3.5-27B | 19/48 | 37/48 | 24/48 | −13 |

Table 11: One consolidation step (50 items $\rightarrow$ 1 super-item) costs every rollout 6–13 wins, with the largest absolute drop at the largest rollout (Qwen3.5-27B, −13). Across all three sizes the post-collapse memory still sits above the no-memory baseline, so the collapse degrades the memory’s value but does not zero it out. Stage 168 is the last pre-collapse stage; stage 169 is the post-collapse single-item state. wins/48 on the balanced ALFWorld split, $n=48$ episodes per cell, single run.

Stage 169 memory (after collapse, 1 item, 1,960 chars) — verbatim

Item 0. Use a single, repeatable loop to handle both single and multi-object tasks across cleaning, cooling, heating, and look-at-in-light. ⁽¹⁾ 1) Systematically search common room sources (kitchen countertop, fridge, cabinet, drawer, dining table, living surfaces) to locate each target; if a known source can hold multiple copies, reuse it for subsequent objects to avoid re-search. 2) Pick each target; if a state change is required (cool X with fridge, heat X with microwave, or clean X with sinkbasin), perform it while holding or immediately after acquisition; open appliances early as needed. 3) Navigate to the destination surface or container; if the destination is a container, open it before placing. 4) Place the object; for multi-object tasks, repeat for each item. 5) If two identical objects must go to the same destination, fetch Object1 from a known multi-copy source and place it, then fetch Object2 from a similarly likely multi-copy source (prefer reusing the same source) and place it at the same destination; carry both if inventory permits. 6) If an object starts at the destination (or inside a container), re-pick it and place again after applying any necessary state change to ensure task closure. 7) Manage distractors by moving unused tools aside to free space; batch carries when possible and reuse multi-copy sources for faster throughput. 8) After completing a batch, return to a known multi-copy source for the next run. 9) Look-at-in-light-specific extension: anchor a central desk lamp location, co-locate the target and lamp by moving one to the other’s surface if needed, activate the lamp, and examine/hold the object under illumination; batch look-at tasks using the same lamp position and multi-copy sources when possible. ⁽²⁾ 10) Pitfalls to avoid: forgetting to re-pick items that started on the destination; forgetting to reopen microwaves or containers when needed; forgetting to retrieve heated/cooled/cleaned objects before placing.

Collapse modes

- (1) **Over-merge into a single “unified loop”.** The 50 structured items of stage 168 each described one decision-relevant insight (task taxonomy, state-change shortcuts, two-object recipes, look-at protocol, etc.). Stage 169 replaces all of them with a single 10-step procedure framed as “a single, repeatable loop to handle both single and multi-object tasks across cleaning, cooling, heating, and look-at-in-light”. Distinctions that were separate items in stage 168 (e.g., the cooling-vs-heating asymmetry, the inventory-permitting two-object shuttle, the lamp-co-location requirement) are reduced to numbered sub-steps with no inter-step preference, no failure cases, and no domain conditions.
- (2) **All task-type-specific procedures collapsed into one item.** Stage 169’s sub-step 9 encodes the entire look-at-in-light protocol (lamp anchoring, co-location, illumination, batching) as a single sub-clause inside an item that primarily reads as a generic pick-and-place loop. Look-at-in-light tasks (8/48 of the eval split) now share retrieval bandwidth with a memory whose lead framing is unrelated. The same compression pattern occurs for cleaning, cooling, and heating: each becomes a half-sentence inside sub-step 2.
- (3) **Magnitude scales with rollout strength.** In table 11 the absolute win drop at $S=168 \rightarrow 169$ grows monotonically with rollout size (−6 for Qwen3.5-4B, −10 for 9B, −13 for 27B; $n=3$ sizes, single run, no significance test). One reading is that stronger rollouts extract more from the structured 50-item memory and therefore lose more when those items are merged into one. The post-collapse memory still beats no-memory for every size, so the collapse removes most of the consolidation’s value but does not destroy it entirely.

G.3. Useless memory items

We distinguish two failure modes for memory items that are syntactically well-formed but supply no decision-relevant information. Both modes survive across stages because nothing contradicts them, and both consume retrieval bandwidth without shifting the action distribution.

Overgeneralized. Items whose recommendation applies to any state (“be careful”, “consider all options”, “plan ahead”). The clause names no observable cue and shifts no prior over candidate actions when retrieved.

We pull three verbatim items from an ExpeL iterative consolidation run on ALFWorld ($n=64$ trajectories per task type, gpt-5.4 base model, gpt-5-nano management, 200 mini-batch consolidation stages) at stage 200. The stage-200 memory has 9 active rules; we display the top-1, top-2, and a low-vote item (Item 7, vote = 2; the lowest is Item 8 with vote = 1).

Overgeneralized memory items (ExpeL ALFWorld stage 200, verbatim)

- Item 0 (vote = 99).** Prioritize actions that directly advance the current subgoal⁽¹⁾ using the nearest feasible fixture or item⁽²⁾; verify prerequisites before acting; minimize travel; interleave subgoals when beneficial⁽³⁾; attempt to combine state-change and final placement in a single near-term step when possible; avoid assuming an item’s state without explicit verification.
- Item 1 (vote = 48).** Maintain a lightweight explicit world model of item locations, containment relationships, and state attributes across environments; update it after every action; verify prerequisites before use; use the model to guide planning, detect contradictions, and resolve ambiguities when multiple identical items exist; keep post-action states consistent for robust planning.
- Item 7 (vote = 2).** Prefer reversible actions and plan with contingencies; always consider alternative paths if the primary plan might fail, to minimize backtracking and avoid dead ends.

Overgeneralization modes

- (1) **Tautological directive.** An agent already pursuing a subgoal already prefers actions that advance it; the clause names no observable cue and so leaves the action distribution unchanged at retrieval.
- (2) **Vacuous ranking criterion.** “Nearest feasible fixture or item” offers a preference but no operational definition of feasibility or distance, no rule for selecting among candidates that all look feasible, and no fallback when none is.
- (3) **Meta-directive without operationalization.** “Interleave subgoals when beneficial” is a control-flow recommendation with no condition for “beneficial”; the item supplies no procedure that produces the benefit signal at action-selection time.

Worked example: how an ExpeL highest-voted insight collapsed. The same run produced Item 0 above by repeatedly applying ExpeL’s EDIT operation to the top slot. By stage 200 the slot had accumulated 99 AGREE/EDIT votes, which would naively suggest extreme value. The vote counter, however, increments on EDIT but is not reset when an EDIT replaces the underlying concept. The score therefore tracks slot edit-volume rather than current content quality. Tracing the slot through stages, its content was rewritten through at least three distinct concepts (section G.3).

ExpeL top-1 slot content drift across stages (verbatim)

Stage 0 (vote = 28). For tasks phrased as “examine/look at X with/by/using Y,” first locate both objects, then prefer the environment’s direct task-relevant interaction (e.g., use Y, examine X with Y, or examine X if Y has been activated); only move objects if explicit admissible actions or clear failure show that relocation is required.
→ *Concrete: explicitly names the `look_at_obj_in_light` examine-with-Y interaction pattern.*

Stage 80 (vote = 46). Prioritize the simplest action that directly advances the current subgoal with the fewest prerequisites and minimal detours, while also favoring actions that reuse already-opened receptacles or inventory-held items to minimize movement; also consider steps that reduce overall future effort.
→ *Generic planning heuristic; the task-type cue is gone.*

Stage 200 (vote = 99). Prioritize actions that directly advance the current subgoal using the nearest feasible fixture or item; verify prerequisites before acting; minimize travel; interleave subgoals when beneficial; attempt to combine state-change and final placement in a single near-term step when possible; avoid assuming an item’s state without explicit verification.
→ *Tautology; applies to any agent benchmark.*

The slot’s 99 votes were earned across three different concepts (examine-with-Y interaction → generic action selection → tautology). At stage 200 only the third concept is in the slot; the other ~ 80 votes were accumulated by predecessors that the EDIT operation has since overwritten. The vote score is therefore not a measure of the current rule’s value to the agent.

Overspecified. Items so narrowly tied to a past trajectory state or to one (object, surface, appliance) tuple that they cannot transfer once that condition is not present. They survive in the cap because nothing contradicts them; they take up retrieval bandwidth on tasks they do not match.

We pull three verbatim items from an ACE iterative consolidation run on ALFWorld (6 abstracts sampled flat-randomly across all six task types per stage; $n=128$ trajectories per type; analyzer dedup threshold 0.75; both reflector and curator = gpt-5.4-mini). Each item is an ADD operation recorded in the curator’s operation log. Reflector and curator are the two ACE roles: the reflector emits bullet candidates from each trajectory, the curator decides which are added to or modified in the running memory, and the analyzer merges any pair of items whose embedding similarity exceeds the dedup threshold.

Overspecified memory items (ACE ALFWorld curator ADDs, verbatim)

Item A. In `look_at_obj_in_light` tasks, follow the environment’s explicit affordances to complete the setup: if the vase has been moved onto the desk, turn on the desk lamp, then verify the vase is on the desk and finish with the available ‘take vase from desk’ action. ⁽¹⁾ Treat the final take-from-desk step as the success-confirming move rather than assuming relocation alone is enough.

Item B. For `pick_heat_then_place_in_recep` tasks, once the target mug is confirmed hot, stop doing additional checks or searches and immediately place/move it into the target receptacle. ⁽²⁾ If the mug is already in hand and hot, the completion action is usually a direct ‘put/place mug in cabinet’ step; do not reopen the microwave, re-verify the temperature, or look for another intermediary action.

Item C. For `pick_cool_then_place_in_recep` tasks involving a mug, prioritize exposed-surface search for the exact mug first, then perform one supported cooling action with the fridge, immediately confirm the same mug is still in inventory, and go straight to the already-open cabinet for final placement. ⁽³⁾ Do not keep searching after the correct mug is found, and do not return to the fridge once cooling has succeeded unless the environment explicitly says the cool action failed.

Overspecification modes

- (1) **Single-instance object+surface pinning.** Item A names two specific object instances (vase, desk) and one specific completion action (take vase from desk). The action sequence does not transfer to a look-at task with a different target (e.g., cd on shelf) because both the object handle and the surface change.
- (2) **Single-receptacle completion pinning.** Item B fixes the heating appliance to microwave and the destination of `pick_heat_then_place` to cabinet. ALFWorld’s actual destinations for this task type include countertop, fridge, sidetable, shelf, etc.; the rule’s put/place mug in cabinet phrasing fires only on (mug, cabinet) instances.
- (3) **Compound condition over object+appliance+destination.** Item C requires (mug and fridge and an already-open cabinet) for its action sequence to apply. Each conjunct narrows the matching set further; the resulting rule fires on a tiny subset of the eval split and contributes nothing on tasks that do not involve all three.

H. WebShop

H.1. Memory artifacts from prior methods

This section reproduces, verbatim, two memory artifacts from prior consolidation methods on WebShop (Yao et al., 2022). They are intended as concrete reference points for the failure-mode taxonomy in section G — not as quantitative comparisons. For each artifact, report: method version and configuration (model, rewrite cadence), WebShop split and seed, and the trajectory step or consolidation round at which the snapshot was taken.

H.1.1. Healthy snapshots: ACE and Dynamic Cheatsheet

We display one mid-run snapshot per method, taken after enough consolidation has happened to populate the memory but before any visible degradation. Both runs use $n=64$ trajectories from the golden WebShop set and gpt-5.4-mini as the consolidation model.

ACE healthy. ACE keeps a flat list of bullet items grouped into named sections (COMMON MISTAKES TO AVOID, PROBLEM-SOLVING HEURISTICS). At stage 100 of a 192-stage run with dedup threshold 0.75, the memory contains 32 bullets totalling about 14k characters. We reproduce one short bullet from each section.

ACE on WebShop, stage 100 (32 bullets, ~14k chars) — representative items

Section: Common mistakes to avoid. Do not invent a reasoning error when the predicted action and ground truth are the same and the environment confirms success. In that case, the correct diagnosis is that no substantive error occurred.

Section: Problem-solving heuristics. For apparel tasks that specify both fit type and size, treat them as a linked final gate: confirm the requested fit type (e.g., youth) and the exact size (e.g., x-small) on the product page before purchase, along with color and price, rather than validating each in isolation.

Dynamic Cheatsheet healthy. Dynamic Cheatsheet keeps a single living document and rewrites it after every trajectory. The document at stage 100 of the same run is roughly 8.3k characters, organised under a `ACTION PATTERNS AND INTERACTION SEQUENCES` heading with one `<memory_item>` per consolidated insight (each with `<description>`, `<example>`, and a `Count` field). We reproduce two consecutive `<memory_item>` blocks.

Dynamic Cheatsheet on WebShop, stage 100 (~8.3k chars) — two-item blocks, verbatim

`<memory_item>`

`<description>`

For apparel tasks with multiple exact requirements, search broad by category plus the most discriminative attributes first, then verify the remaining constraints on the product page. Do not trust title-only matches for fabric, fit, care, color, size, or price. (Reference: current input)

`</description>`

`<example>`

Template:

- 1) Search: men's dress shirt cotton spandex classic fit short sleeve machine wash melon berry 6x under 60
- 2) Open best-looking product
- 3) Verify category = men's dress shirt
- 4) Verify fabric = cotton spandex
- 5) Verify fit = classic fit
- 6) Verify sleeve = short sleeve
- 7) Verify care = machine wash
- 8) Verify color = melon berry
- 9) Verify size = 6x
- 10) Verify price < 60.00
- 11) Buy only after all checks pass

`</example>`

**** Count: 1**

`</memory_item>`

`<memory_item>`

`<description>`

Exact color variants are non-substitutable. If the task requests a precise color name like “melon berry,” do not accept nearby pink/red/coral shades, generic berry tones, or other “melon” variants unless the swatch label matches exactly. (Reference: current input)

`</description>`

`<example>`

Accept only:

- melon berry

Reject nearby variants:

- berry
- melon
- rose

```
- coral
- salmon
- watermelon
- dusty rose
- light red
</example>
** Count: 1
</memory_item>
```

These two snapshots illustrate the methods’ design intent: ACE produces structured bullet lists with explicit failure-mode categories, while Dynamic Cheatsheet produces one continuously-edited document with embedded action templates and counter-examples. Both show concrete domain content (apparel, sizes, colour names, price thresholds) and avoid the empty-content / paraphrase failure modes of sections G.1 and G.3.

H.1.2. Collapsed memory: Agent Workflow Memory

Setup. The artifact is the AWM-style consolidated workflow file at *epoch* 3 of the same training run that supplies the epoch-1 snapshot used in the W8 ablation of section I.1: 128 golden WebShop trajectories ($n=128$, all reward = 1.0), gpt-5.4-mini consolidator, batch size 16, 3 epochs. Epoch 1 contains 8 workflows (W1–W8 in section I.1); epoch 3 contains 16 workflows (W1–W16 below). The first 8 workflows of epoch 3 share titles and substantive content with epoch 1 (W2–W6 and W8 are character-identical modulo trailing whitespace; W1 and W7 receive minor wording edits over the next two epochs). The additional W9–W16 are new entries the consolidator appended over those epochs.

The collapsed snapshot. We list epoch 3’s 16 workflow titles below. The eight W9–W16 entries (highlighted) are category-specific specialisations of W1–W7; none introduces a new control-flow pattern or stop criterion not already present in W1–W7. We treat this as collapsed by accretion: the cap of the consolidation pipeline does not bind here, but the consolidator nevertheless stores redundant restatements of the same templates.

AWM on WebShop, epoch 3 (16 workflows, ~8.2k chars) — workflow titles, verbatim

- W1. Search by attribute-rich query.
- W2. Open candidate item to inspect options.
- W3. Select required attributes before buying.
- W4. Select required size, color, and other variant options before buying.
- W5. Search and select clothing variants with fit type.
- W6. Search and select home decor variants.
- W7. Search and select multi-part apparel sizing variants.
- W8. Search across pages when the first results do not match.
- W9. Select apparel color, size, and fit/order-specific variants before buying. ⁽¹⁾
- W10. Select non-apparel flavor and size variants before buying. ⁽²⁾
- W11. Select pack-count and color variants before buying. ⁽²⁾
- W12. Search and select color, size, and shape variants for home goods. ⁽²⁾
- W13. Search and select shoes with color and size. ⁽²⁾
- W14. Search and select apparel color, size, and purchase. ⁽²⁾
- W15. Search and select electronics memory/storage variants. ⁽²⁾
- W16. Search and select activewear or performance tops variants. ⁽²⁾

Collapse modes

- (1) **Re-derivation of an existing workflow.** W9 “Select apparel color, size, and fit/order-specific variants before buying” is the union of W3 (“Select required attributes before buying”), W4 (“size, color, and other variant options”), and W5 (clothing fit type) restated in one entry. The action sequence inside W9 reuses the same primitives (`click[<required_color>]`, `click[<required_size>]`, etc.) as those three predecessors; no new primitive or guard is added.
- (2) **Per-category specialisations of W1–W4.** W10–W16 each pin the search-then-variant-select template (W1–W4) to one product category (food/flavour, pack-count, home goods, shoes, apparel, electronics, activewear). Their action sequences differ only in which placeholder names appear inside `search[...]`; the control flow is identical to W2 followed by W3/W4. The eight new entries shorten the retrieval bandwidth available for the eight original abstract workflows without adding any control-flow generality. The W8 page-turn pathology of section I.1 persists unchanged in epoch 3.

I. Memory surgery

We isolate the contribution of individual memory items to overall failure by removing one item from a memory snapshot and re-evaluating, holding the rollout policy and all other items fixed.

I.1. Workflow 8 ablation on WebShop

Setup. The memory under study is a consolidated workflow file (8 workflows W1–W8, reproduced verbatim in section I.1) produced after one epoch of AWM-style consolidation. AWM-style consolidation abstracts each input trajectory into a workflow template that pairs a thought rationale with an action sequence; we ablate Workflow 8 (W8) of the resulting file. The training data are 128 expert (task reward 1.0) trajectories sampled from the AgentGym AgentTraj-L WebShop corpus (seed 42). The eval split is a 50-session subset of WebShop’s test pool (seed 42) with overlap to the training sessions removed. We construct two evaluation conditions on this split:

- **Full memory:** all 8 workflows.
- **Memory minus W8:** the 7 workflows from the same file with Workflow 8 removed (no other edits).

All conditions use `eval_max_steps=30`, `eval_max_tokens=4096`, and `eval_num_runs=1`. We do not set a temperature parameter. Each condition is evaluated under two rollout policies, `gpt-5.4-mini` and `gpt-5-mini`. The memory artifact is byte-identical across the two rollouts.

The WebShop action space used in table 13 groups raw `click[...]` actions into six categories by the type of UI element clicked:

- **search:** issue a query against the catalog, e.g., `search[red running shoes size 10 < $60]`. Returns a paginated list of product cards.
- **click ASIN:** open a product detail page from a search-result card by its ASIN (Amazon Standard Identification Number, a 10-character product code such as `b09npml43m`), e.g., `click[b09npml43m]`. This is the only way to inspect a product’s variants, price, attributes, and Buy Now button.
- **click variant:** select a required option on a product page before purchase, e.g., `click[medium]`, `click[b16-red]`. Many products require one or more such selections; missing a required variant prevents reward 1.0 even if Buy Now succeeds.
- **click [Next >]:** advance to the next page of search results without opening any product, e.g., `click[Next >]`, `click[next >]`. We also count the symmetric paging primitives `click[< Prev]` here.
- **click back:** return to the search input from a results or product page, e.g., `click[Back to Search]`, abandoning the current query.
- **click [Buy Now]:** commit to purchasing the currently configured product. Terminates the episode and yields the final reward in $[0, 1]$ based on attribute match against the instruction.

The consolidated memory. The full saved memory file contains 8 workflows, W1–W8, reproduced verbatim below. The original file delimits each block with XML-style `<think>` and `<action>` tags; we render them as italic “Thought”/“Action sequence” labels for readability and otherwise preserve the text. Highlighted spans in W8 key into the paired annotations callout that follows.

Consolidated memory file: 8 workflows W1–W8 (verbatim)

Workflow 1: Search by attribute-rich query

Thought. Build a single search query that combines the headline category words plus the most distinguishing attributes, variant-like descriptors, and the price cap. Use the instruction’s key terms so WebShop can match the product listing by keyword overlap.

Action sequence. `search[<headline_category> <key_attribute_1> <key_attribute_2> ... <required_color> <required_size> price < <price_cap>]`

Workflow 2: Open candidate item to inspect options

Thought. After a search, click the best-matching ASIN-like result to open the product detail page where color/size pickers or other variants are exposed.

Action sequence. `click[<candidate_item_id>]`

Workflow 3: Select required attributes before buying

Thought. On the product detail page, click each required variant option from the instruction, such as color and size, in any order before purchasing.

Action sequence.

`click[<required_color>]; click[<required_size>]; click[Buy Now].`

Workflow 4: Select required size, color, and other variant options before buying

Thought. Some products expose multiple selectable variants beyond color and size, such as fit type, style, pack count, flavor, or another categorical option embedded in the product page. Click every required option from the instruction before buying.

Action sequence.

`click[<required_variant_1>]; click[<required_variant_2>];
click[<required_variant_3>]; click[Buy Now].`

Workflow 5: Search and select clothing variants with fit type

Thought. For apparel that specifies fit type along with color and size, search with the apparel category and all major descriptors, then open the best result and select fit type, color, size, and buy.

Action sequence.

`search[<apparel_category> <key_descriptors> <required_color> <required_fit_type>
<required_size> price < <price_cap>]; click[<candidate_item_id>];
click[<required_fit_type>]; click[<required_color>]; click[<required_size>];
click[Buy Now].`

Workflow 6: Search and select home decor variants

Thought. For home decor items such as window coverings or lighting, search using the room/use-case keywords plus the product descriptors and price cap, then open the best result and select the requested size or color before buying.

Action sequence.

`search[<home_decor_category> <key_descriptors> <room_or_use_case> <required_color>
<required_size> price < <price_cap>]; click[<candidate_item_id>];
click[<required_size>]; click[<required_color>]; click[Buy Now].`

Workflow 7: Search and select multi-part apparel sizing variants

Thought. Some apparel sizes include compound labels like tall, big, or tag-based sizes. Search with the product descriptors and the base size text, open the item, then choose the exact compound size and other required options before buying.

Action sequence.

```
search[<apparel_category> <key_descriptors> <required_color> <base_size_text>
price < <price_cap>]; click[<candidate_item_id>]; click[<required_color>];
click[<required_compound_size>]; click[Buy Now].
```

Workflow 8: Search across pages when the first results do not match

Thought. If the initial result page does not contain a plausible match, continue to the next page and inspect additional candidates until a suitable item appears, ⁽¹⁾ then open it and proceed with variant selection and purchase.

Action sequence.

```
click[Next >] (3); click[<candidate_item_id> (2)]; click[<required_variant_1>];
click[<required_variant_2>]; click[Buy Now].
```

Why this memory is harmful

- (1) **No stop criterion.** The natural-language description tells the agent to continue paging until a “suitable item appears,” but suitability is not defined relative to any concrete feature of the agent’s task. The agent therefore has no signal for when to halt the loop.
- (2) **Abstract candidate placeholder.** In W2 and W5–W7 of section I.1, click[<candidate_item_id>] is preceded by an “open the best-matching result” step (highlighted in green in those workflows) that picks a concrete ASIN. W8 omits this selection step, so the placeholder is never bound to a real product.
- (3) **Page-turn primitive promoted to a workflow.** click[Next >] is a low-level navigation action. It appears nowhere in the action sequences of W1–W7. W8 elevates it to a first-class workflow, signalling to the agent that turning pages without committing is a sanctioned strategy.

Result. table 12 reports per-condition outcomes; table 13 breaks down the action stream by category.

Table 12: Removing W8 yields higher wins, higher mean reward, and lower mean steps in both rollouts; even so, gpt-5.4-mini reaches only 14/50 wins after removal, so the bulk of failures under this rollout is not attributable to W8 alone. Under gpt-5.4-mini no episode reaches the 30-step cap in either condition; the cap reduction is concentrated in gpt-5-mini (22 → 17). Per-condition outcomes for the two rollout policies, n=50 episodes per cell, single run. Columns: episodes solved out of 50; mean per-episode reward in [0, 1]; mean trajectory length in steps; episodes that hit the max steps (which is 30).

| Rollout | Memory | wins/50 | mean reward | mean steps | hit step cap |
|--------------|------------------------|---------|-------------|------------|--------------|
| gpt-5.4-mini | full (8 workflows) | 7/50 | 0.23 | 12.4 | 0/50 |
| gpt-5.4-mini | minus W8 (7 workflows) | 14/50 | 0.37 | 11.4 | 0/50 |
| gpt-5-mini | full (8 workflows) | 18/50 | 0.49 | 18.2 | 22/50 |
| gpt-5-mini | minus W8 (7 workflows) | 23/50 | 0.59 | 15.8 | 17/50 |

Behavioral evidence. With W8 in context, `click[Next >]` dominates the action stream relative to product-page interactions and `click[Buy Now]`. Removing W8 reduces this concentration in both rollouts ($n=50$ episodes per cell, single run; no significance test). Under gpt-5.4-mini, observed page-turn counts are 421 (full) and 181 (minus W8), a ratio of $2.3\times$; the count of distinct episodes that issue at least one `click[Buy Now]` is 14/50 (full) and 21/50 (minus W8). Under gpt-5-mini, page-turn counts are 559 and 246 (also $2.3\times$); the Buy-Now-episode count is 28/50 and 33/50.

Table 13: With W8 in context, `click[Next >]` dominates the action stream relative to product-page interactions and `click[Buy Now]`; removing W8 reduces this concentration in both rollouts (page-turn ratio $2.3\times$ full/minus-W8 in both). Action-category counts over $n=50$ episodes per cell, single run. Action categories are defined in the bullet list of section I.1. “Buy Now (eps)” is the number of distinct episodes that issued at least one `click[Buy Now]`.

| Action | gpt-5.4-mini rollout | | gpt-5-mini rollout | |
|-----------------|----------------------|----------|--------------------|----------|
| | Full memory | Minus W8 | Full memory | Minus W8 |
| search | 81 | 142 | 117 | 183 |
| click ASIN | 42 | 72 | 70 | 97 |
| click variant | 28 | 55 | 71 | 99 |
| click [Next >] | 421 | 181 | 559 | 246 |
| click back | 33 | 97 | 67 | 133 |
| click [Buy Now] | 14 | 21 | 28 | 33 |
| Buy Now (eps) | 14 | 21 | 28 | 33 |

J. ScienceWorld

J.1. Abnormal memories

J.1.1. Representative cases

We first present three representative abnormal memory cases identified in the ScienceWorld environment, one from each abnormality category. These cases illustrate qualitatively different failure modes in memory consolidation: `over_generalization`, where a memory abstracts from prior experience too broadly and may bias the agent toward inappropriate actions; `over_specialization`, where a memory is tied to overly specific entities or procedures and therefore has limited transferability; and `useless_garbage`, where a memory is vague, malformed, or insufficiently actionable. For each case, we annotate the problematic components of the memory and explain why retrieving it may hurt downstream decision-making.

Representative `over_generalization` memory from ScienceWorld

Using a lighter, fire source, or oven MAY BE NECESSARY⁽¹⁾ to change the state of a food or substance⁽²⁾ in state-change tasks⁽³⁾.

Why this memory is harmful

- (1) **Overgeneralized tool prior.** The memory promotes a broad prior over heat sources, even though many state-change tasks may require cooling, freezing, melting, or task-specific apparatus rather than a lighter, fire source, or oven.
- (2) **Underspecified target transformation.** “Change the state” collapses different physical transformations into one generic goal, ignoring whether the required transition is boiling, freezing, melting, or another state change.

- (3) **Cross-task overreach.** The rule is stated for all state-change tasks, so retrieval may bias the agent toward heat-related actions even in contexts where heating is irrelevant or harmful.

Representative over_specialization memory from ScienceWorld

For conductivity-sorting tasks, picking up and testing unknown substance B⁽¹⁾ in a circuit SHOULD BE NECESSARY before placing it in the correct box⁽²⁾.

Why this memory is harmful

- (1) **Instance-specific binding.** The memory is tied to “unknown substance B” rather than the general class of unknown substances, so it may fail to transfer to otherwise identical tasks involving different item names.
- (2) **Narrow task schema.** The final action is framed around a specific sorting protocol, making the memory useful only for a narrow subset of conductivity tasks rather than a robust reusable strategy.

Representative useless_garbage memory from ScienceWorld

Using the correct container or setup⁽¹⁾ MAY BE CONTRIBUTE to boiling the target substance⁽²⁾.

Why this memory is harmful

- (1) **Vague object reference.** “Correct container or setup” does not specify which observable object should be selected, moved, opened, filled, or heated, so it provides little actionable guidance.
- (2) **Generic task hint.** The memory merely restates that some appropriate setup may help with boiling, but does not identify a concrete precondition, action sequence, or failure mode.

J.1.2. Additional examples

We further list additional abnormal memories selected by the LLM-as-judge from ScienceWorld memory banks. These examples are grouped into the same three categories: over_generalization, over_specialization, and useless_garbage.

Abnormal Memories: over_generalization

[over_generalization #1]
Memory: Observing all life stages in order MAY BE NECESSARY to complete a turtle-stage task.

[over_generalization #2]
Memory: Observing all life stages in order MAY BE NECESSARY to complete a moth-stage task.

[over_generalization #3]
Memory: Using a lighter or fire source MAY BE NECESSARY to change the state of matter of water in outdoor state-change tasks.

[over_generalization #4]
Memory: Using a lighter, fire source, or oven MAY BE NECESSARY to change the state of a food or substance in state-change tasks.

[over_generalization #5]
Memory: Focusing on the target substance SHOULD BE NECESSARY to change its state.

Abnormal Memories: over_specialization

[over_specialization #1]

Memory: For conductivity-sorting tasks, picking up and testing unknown substance B in a circuit SHOULD BE NECESSARY before placing it in the correct box.

[over_specialization #2]

Memory: MAY CONTRIBUTE to anchoring progress checks to observable context (current room, visible objects) before initiating life-stage sequencing in melting-point tasks, where initial observations (thermometer, boxes) must be secured before progress.

[over_specialization #3]

Memory: For salt water recipes, placing salt and a cup/container in the sink MAY BE NECESSARY before pouring water.

[over_specialization #4]

Memory: For sodium acetate recipes, moving the required ingredients into a glass cup MAY BE NECESSARY before focusing on the product.

[over_specialization #5]

Memory: SEED_ACQUISITION_BARRIER: Access to viable apple seeds often hinges on physically locating seeds in connected environments (e.g., kitchen) and then transporting them to an appropriate growing area.

Abnormal Memories: useless_garbage

[useless_garbage #1]

Memory: Assembling an electrical circuit DOES NOT CONTRIBUTE to completing a smores synthesis task.

[useless_garbage #2]

Memory: Mixing the correct ingredients MAY BE NECESSARY to create a target solution or mixture.

[useless_garbage #3]

Memory: Mixing ingredients in a glass cup MAY BE NECESSARY to create smores after reading the recipe.

[useless_garbage #4]

Memory: Applying the correct cultivation action after planting a seed MAY BE NECESSARY to advance plant growth tasks.

[useless_garbage #5]

Memory: Using the correct container or setup MAY BE CONTRIBUTE to boiling the target substance.

[useless_garbage #6]

Memory: Moving the target container to a working oven MAY BE CONTRIBUTE to boiling the target substance.

[useless_garbage #7]

Memory: Repeated heating actions MAY BE CONTRIBUTE to gradually raising the temperature of the target substance.

[useless_garbage #8]

Memory: Using a lighter MAY BE CONTRIBUTE to boiling a substance when combustion is allowed.

[useless_garbage #9]

Memory: Opening and using an oven MAY BE CONTRIBUTE to boiling or heating a target substance.

[useless_garbage #10]

Memory: Waiting after activating a heat source MAY BE CONTRIBUTE to changing the state of a target substance.

[useless_garbage #11]

Memory: Opening an oven, placing the target substance inside, and waiting MAY BE CONTRIBUTE to changing the state of matter of the target substance.

[useless_garbage #12]

Memory: Focusing on the target substance MAY BE NECESSARY to change its state.

[useless_garbage #13]

Memory: Focusing on the wrong substance DOES NOT CONTRIBUTE to changing the state of the target substance.

J.1.3. Memory banks across consolidation rounds

We further examine the evolution of memory banks produced by GPT-5.4-mini in the ScienceWorld environment. The model performs 20 rounds of memory consolidation over 485 rollouts. We show three representative memory banks: **the first round**, which reflects the initial memory bank; **the tenth round**, which reflects an intermediate memory bank; and **the twentieth round**, which reflects the final memory bank. For each memory bank, we also report the corresponding average test score.

Initial memory bank: round 1, Memory Bank Size = 16, average test score = 7.45

[memory #1]

Life-stage identification tasks MAY BE NECESSARY to search the relevant rooms for the specified organism stages in order; wandering through unrelated rooms and inspecting irrelevant objects DOES NOT CONTRIBUTE to success.

[memory #2]

Sandwich-making tasks MAY BE NECESSARY to read the recipe, gather the listed ingredients, and mix them in the correct container; opening empty drawers or collecting unrelated items DOES NOT CONTRIBUTE to success.

[memory #3]

Conductivity tasks MAY BE NECESSARY to build the circuit, observe the switch result, and then place the substance in the box matching conductivity; repeated reconnect attempts on already-connected parts DOES NOT CONTRIBUTE to success.

[memory #4]

Plant-growth tasks MAY BE NECESSARY to place the seed in soil or a pot, water it if required, and then wait for growth; pouring seeds into the agent or moving them between containers DOES NOT CONTRIBUTE to success.

[memory #5]

State-change tasks MAY BE NECESSARY to use the task-specific tool or heat/cool source on the target substance and wait; interacting with unrelated objects or empty containers DOES NOT CONTRIBUTE to success.

[memory #6]

Melting-point tasks MAY BE NECESSARY to measure the substance with a thermometer before choosing the box; focusing on the wrong box after a valid reading DOES NOT CONTRIBUTE to success.

[memory #7]

Freezing tasks MAY BE NECESSARY to put the target substance in a freezer or otherwise cool it, then wait for the state change; wandering without locating the substance DOES NOT CONTRIBUTE to success.

[memory #8]

Paint-mixing tasks MAY BE NECESSARY to combine the required colored paints in the correct container; repeatedly pouring a single color or empty containers DOES NOT CONTRIBUTE to success.

[memory #9]

Life-stage identification tasks MAY BE NECESSARY to search the relevant rooms for the specified organism stages in order; wandering through unrelated rooms and inspecting irrelevant objects DOES NOT CONTRIBUTE to success.

[memory #10]

Sandwich-making tasks MAY BE NECESSARY to read the recipe, gather the listed ingredients, and mix them in the correct container; opening empty drawers or collecting unrelated items DOES NOT CONTRIBUTE to success.

[memory #11]

Some conductivity tasks MAY BE NECESSARY to use the correct circuit components and then sort the substance into the box matching the observed conductivity; testing with unrelated objects or looping on already-connected terminals DOES NOT CONTRIBUTE to success.

[memory #12]

Melting-point tasks MAY BE NECESSARY to measure the target substance before choosing the box; focusing on the wrong box after a valid reading DOES NOT CONTRIBUTE to success.

[memory #13]

Boiling/combustion tasks MAY BE NECESSARY to use the stove if it works, but if the stove is broken a lighter MAY BE NECESSARY to ignite the target substance.

[memory #14]

Motor-power tasks MAY BE NECESSARY to connect the generator, wires, and motor correctly before activating the power source.

[memory #15]

Ice-cream heating tasks MAY BE NECESSARY to move the target into an oven when the stove is broken, then wait for completion.

[memory #16]

Dough-making tasks MAY BE NECESSARY to combine flour and water; pouring unrelated items or empty containers DOES NOT CONTRIBUTE to success.

Intermediate memory bank: round 10, Memory Bank Size = 10, average test score = -7.68

[memory #1]

Recipe-based mixing tasks MAY BE NECESSARY to read the recipe first, gather the exact listed ingredients, and mix them in the intended container; unrelated pouring, extra items, or mixing in the wrong place DOES NOT CONTRIBUTE to success.

[memory #2]

Plant-growth tasks MAY BE NECESSARY to keep the seed in the greenhouse or other growth area with soil or a pot and water while waiting; moving seeds through cups, jars, toilets, or the agent DOES NOT CONTRIBUTE to growth.

[memory #3]

Temperature-sorting tasks MAY BE NECESSARY to locate both the thermometer and the target substance, measure first, and only then choose the correct box; selecting a box before obtaining a reading or wandering without measuring the target DOES NOT CONTRIBUTE to success.

[memory #4]

State-change tasks MAY BE NECESSARY to place the target substance in the correct heating or freezing setup, then wait long enough and verify the transformed state; skipping the wait or focusing on unrelated objects DOES NOT CONTRIBUTE to completion.

[memory #5]

Conductivity tasks MAY BE NECESSARY to assemble the circuit with the target object, power source, wires, switch, and indicator before sorting it into the box matching the test result; repeated attempts to connect already-occupied terminals or other connection loops DOES NOT CONTRIBUTE to success.

[memory #6]

Recipe-based mixing tasks MAY BE NECESSARY to read the recipe first, gather the exact listed ingredients, and mix them in the intended container; unrelated pouring, extra items, or mixing in the wrong place DOES NOT CONTRIBUTE to success.

[memory #7]

Plant-growth tasks MAY BE NECESSARY to keep the seed in the greenhouse or other growth area with soil or a pot and water while waiting; moving seeds through cups, jars, toilets, or the agent DOES NOT CONTRIBUTE to growth.

[memory #8]

Conductivity tasks MAY BE NECESSARY to assemble the circuit with the target object, battery, wires, switch, and light bulb, activate the switch, and then place the object into the box matching the observed result; the green box can be the correct destination for a conductive result.

[memory #9]

Temperature-sorting tasks MAY BE NECESSARY to retrieve the thermometer from one room, measure the target substance in another room, and then choose the box based on the threshold; the bathroom can be the final placement room after measuring in the living room.

[memory #10]

State-change tasks MAY BE NECESSARY to heat the target in a stove or pot setup, wait through multiple turns, and only then decide the box; starting the heating setup without completing the final reading leaves the task incomplete.

Final memory bank: round 20, Memory Bank Size = 17, average test score = -8.66

[memory #1]

Some life-stage tasks SHOULD BE NECESSARY to retrieve the seed, plant it in soil or a pot, then wait or water appropriately until the target stage appears; repeated self-pouring or unrelated interactions DOES NOT CONTRIBUTE to success.

[memory #2]

Some recipe tasks SHOULD BE NECESSARY to read the recipe, gather the exact listed ingredients, combine them in the required container, and then focus on the created product to finish.

[memory #3]

Some conductivity tasks SHOULD BE NECESSARY to assemble a clean circuit with battery, wires, switch, and bulb, observe the result, and place the substance in the matching box; repeated reconnecting or terminal-occupied errors DOES NOT CONTRIBUTE to success.

[memory #4]

Some paint-mixing tasks MAY BE NECESSARY to use the exact color ingredients and avoid pouring empty cups or unrelated items into the agent.

[memory #5]

Some conductivity tasks MAY BE NECESSARY to avoid repeated connection loops and terminal-occupied errors when building the circuit.

[memory #6]

Some plant-growth tasks MAY BE NECESSARY to plant the seed first and then wait or water appropriately; repeatedly pouring the seed into the agent or moving it between inventory and hand DOES NOT CONTRIBUTE to success.

[memory #7]

Some conductivity tasks MAY BE NECESSARY to avoid sorting the substance before the circuit test is complete; premature box placement DOES NOT CONTRIBUTE to success.

[memory #8]

Some life-stage tasks MAY BE NECESSARY to ignore distractor animals or pictures and keep searching for the required species stages.

[memory #9]

Some paint-mixing tasks MAY BE NECESSARY to combine the specified colors directly rather than repeatedly pouring a single color or empty containers.

[memory #10]

Some state-change tasks MAY BE NECESSARY to use the correct heat or cold source on the target, wait, verify the final temperature or state, and then choose the matching box; skipping measurement or choosing the wrong box DOES NOT CONTRIBUTE to success.

[memory #11]

Some state-change tasks MAY BE NECESSARY to use a lighter or oven when the stove is broken, especially for marshmallow, ice cream, or similar items.

[memory #12]

Some freezing tasks MAY BE NECESSARY to place the target in an ultra-low temperature freezer, wait, and then retrieve it; removing it too early MAY BE CONTRIBUTE to incomplete progress.

[memory #13]

Some temperature-box tasks MAY BE NECESSARY to measure the target first and then place it in the box dictated by the reading; failing to do the final placement leaves the task incomplete.

[memory #14]

Some state-change tasks MAY BE NECESSARY to move the target onto a stove or pot, wait, then use the thermometer in the heated container before choosing the box.

[memory #15]

Some state-change tasks MAY BE NECESSARY to heat ice cream in a cup on the stove when the task requires changing its state.

[memory #16]

Some temperature-box tasks MAY BE NECESSARY to measure the target in the kitchen or living room, then carry it to the correct colored box in another room.

[memory #17]

Some combustion tasks MAY BE NECESSARY to use a lighter directly on a marshmallow when boiling is not possible.

K. Additional Analyses

K.1. Auto-Mode Component Ablation

The main-text Auto + Episodic curve in Fig. 9b couples two memory sources. To isolate their contributions we re-run evaluation at four checkpoints with each source restricted in turn: *Abstract Only* reads only the distilled abstract memory, *Episodic Only* reads only the raw episodic store, and *Auto* reads both. The Episodic Only condition recovers nearly all of the Auto gain; Abstract Only never improves on zero-shot (Fig. 17).

K.2. Memory Composition under Task Switching

Supporting evidence for the Fresh-vs-Cumulative comparison in §6.2. After each of the 15 ScienceWorld switch tasks, we ask an LLM judge to label every entry currently in the memory store as *normal*, *over-generalized*, *over-specific*, or *garbage*. Figure 18 plots the running totals of the two failure-mode categories (over-gen and garbage) across the sequence for GPT5.4-mini. Cumulative’s totals diverge from Fresh’s almost immediately and never recover, supplying a direct mechanistic account of the +203-point score gap reported in the main text.

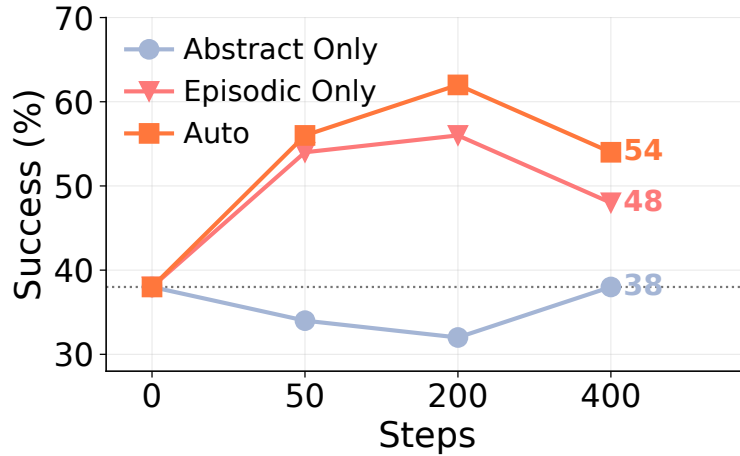


Figure 17: What carries the Auto-mode gain? On the GT-400 stream, we ablate which memory source conditions the solver at four checkpoints: *Abstract Only* uses only distilled abstract entries, *Episodic Only* uses only the raw episodic store, and *Auto* uses both. Episodic Only accounts for nearly all of the gain; Abstract Only stays at or below the zero-shot baseline (dotted) throughout; combining the two adds a few extra points at every checkpoint but the late-step decline at 400 tracks the abstract store’s gradual erosion.

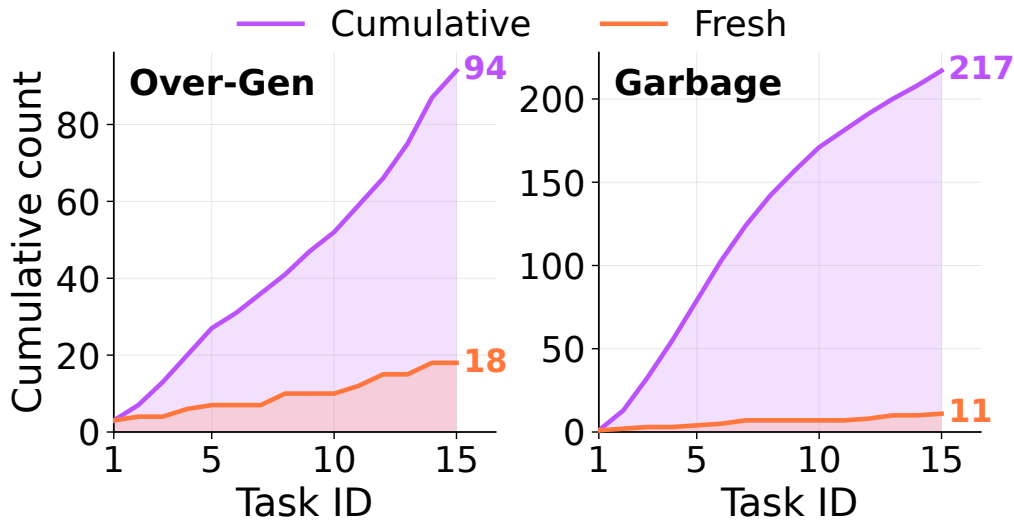


Figure 18: What accumulates in memory under task switching (GPT5.4-mini). Running totals of LLM-judge-flagged over-generalized and garbage entries across the 15-task ScienceWorld switch sequence, comparing *Cumulative* (consolidate over tasks $1..t$) against *Fresh* (consolidate only on task t). Cumulative buries the store in over-gen and garbage memories that Fresh never accrues, with the gap widening monotonically over the sequence. Referenced from §6.2.